

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«ВИТЕБСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ ИМЕНИ
П.М. МАШЕРОВА»

Факультет математики и информационных технологий

Кафедра прикладного и системного программирования

Допущен к защите
«__» _____ 20__
Заведующий кафедрой
_____ Е.А. Корчевская

ДИПЛОМНАЯ РАБОТА

ПРОЕКТИРОВАНИЕ И РЕАЛИЗАЦИЯ WI-FI USB-НАКОПИТЕЛЯ С
ФУНКЦИЕЙ ОБЛАЧНОГО ДОСТУПА К ДАННЫМ

Специальность 1-40 01 01 03 «Программное обеспечение информационных технологий. Базы данных и программное обеспечение информационных систем»

Нестерович Кирилл Анатольевич,
4 курс, группа 22ПОИТ1з

Научный руководитель:
старший преподаватель кафедры
прикладного и системного
программирования,
П.В. Травничева

Витебск, 2026

Утверждаю
Заведующий кафедрой
Е.А. Корчевская
(подпись)
« » 20 г.

Дата « » 20 г.

Реферат

Дипломная работа 48 с., 16 рис., 9 листингов, 53 источника, 2 прил.

USB-НАКОПИТЕЛЬ, БЕСПРОВОДНОЙ ДОСТУП, ВСТРОЕННЫЕ СИСТЕМЫ, ХРАНЕНИЕ ДАННЫХ, ВЕБ-ИНТЕРФЕЙС, МИКРОКОНТРОЛЛЕР, ФАЙЛОВАЯ СИСТЕМА, ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ, СЕТЕВЫЕ ТЕХНОЛОГИИ, АВТОНОМНЫЕ УСТРОЙСТВА, АРХИТЕКТУРА СИСТЕМЫ.

Объект исследования – автономные системы хранения данных с проводным и беспроводным доступом.

Предмет исследования – архитектура и методы реализации USB-накопителя с функцией облачного доступа к данным.

Цель дипломной работы – спроектировать и реализовать USB Wi-Fi накопитель, обеспечивающий локальный доступ к данным через интерфейс USB и удаленный доступ через встроенный Wi-Fi модуль и веб-интерфейс без использования внешних облачных сервисов.

Методы исследования: анализ и обобщение научно-технических источников, проектирование аппаратных и программных систем, тестирование и оценка системы, моделирование режимов работы встроенных систем.

Элементы новизны заключаются в разработке гибридной архитектуры USB-накопителя, совмещающей функции классического съемного носителя и автономного Wi-Fi сервера с веб-интерфейсом управления, а также в реализации механизма безопасного переключения режимов доступа к файловой системе без нарушения целостности данных.

Результаты внедрения: разработка использована в производственном процессе ОАО «Полоцкий молочный комбинат».

Теоретическая и практическая значимость работы заключается в возможности использования полученных результатов при разработке встроенных систем хранения данных, а также в учебном процессе при изучении архитектуры встроенных устройств, сетевых технологий и клиент-серверных взаимодействий.

Оглавление

Введение	3
1 Анализ предметной области и существующих решений.....	6
1.1 Автономные USB-накопители и системы хранения данных	6
1.2 Беспроводной доступ к данным и облачные технологии	7
1.3 Анализ существующих решений и аналогов	8
2 Проектирование и реализация Wi-Fi USB-накопителя.....	11
2.1 Постановка задачи и требования к системе	11
2.2 Общая архитектура устройства	12
2.3 Аппаратная реализация устройства	14
2.4 Архитектура встроенного программного обеспечения.....	16
2.5 Реализация режима USB Mass Storage.....	18
2.6 Реализация беспроводного доступа по Wi-Fi и управления конфигурацией	21
2.7 Реализация встроенного веб-сервера и пользовательского интерфейса.....	25
2.8 Реализация механизма обновления встроенного ПО	29
3 Оценка функционирования и устойчивости системы.....	32
3.1 Методика и условия тестирования устройства	32
3.2 Тестирование проводного режима работы	32
3.3 Тестирование беспроводного режима доступа	34
3.4 Тестирование переключения режимов работы	37
Заключение.....	39
Список использованных источников	42
Приложение А.....	47
Приложение Б	48

Введение

Несмотря на стремительное развитие сетевых и облачных сервисов, USB-накопители по-прежнему широко используются как самый простой способ локального хранения данных. На практике они часто используются для переноса рабочих файлов между несколькими компьютерами или другими устройствами, обновления встроенного программного обеспечения оборудования, прослушивания музыки или просмотра фильмов на мультимедийных устройствах. Обычно такие накопители форматируются в популярных файловых системах FAT32 или NTFS, благодаря чему корректно определяются всеми устройствами и операционными системами без дополнительной настройки. Но при этом доступ к данным остается только при физическом подключении устройства к хост-устройству, что исключает удаленную работу с файлами и не позволяет использовать накопитель одновременно с нескольких устройств без участия кабеля [1].

На первый взгляд альтернативой локальным носителям могли бы стать облачные и сетевые хранилища, которые сегодня используются прежде всего для удаленного доступа к данным через интернет. Такой подход удобен, но на практике он сильно зависит от наличия стабильного соединения и доступности внешних сервисов. При работе в условиях нестабильной сети, а также в изолированных сегментах, например на производственных объектах или в учебных лабораториях, подобные решения оказываются неудобными или вовсе неприменимыми. Дополнительные ограничения связаны с тем, что данные размещаются на сторонних серверах, что не всегда допустимо при работе с конфиденциальной или служебной информацией. Поэтому в ряде инженерных и учебных задач возникает потребность в автономных устройствах хранения, которые позволяют получить беспроводной доступ к данным локально, без использования внешних облачных сервисов [2; 3].

В таких условиях актуальной становится разработка устройств хранения информации, сочетающих возможности стандартного USB-накопителя и облачного доступа к данным [4]. Такие решения позволяют работать с файлами как при физическом подключении к компьютеру, так и через локальную беспроводную сеть, например, с использованием встроенного веб-интерфейса или специального программного обеспечения. Это расширяет возможные сценарии применения хранилища и упрощает работу с данными в таких ситуациях, когда локальное подключение не совсем удобно или вовсе невозможно. При этом наибольший практический интерес представляют устройства, которые могут работать автономно в пределах локальной сети и не зависеть от внешних сервисов или сетей.

На текущий момент на рынке уже существуют различные устройства, ориентированные на создание возможности беспроводного доступа к данным,

например коммерческие USB-накопители с поддержкой подключения к ним по Wi-Fi, SD-карты памяти с модулем беспроводного доступа и специальные адаптеры. Часто такие решения рассчитаны на массового пользователя и поставляются с проприетарным программным обеспечением, а их закрытая архитектура ограничивает возможности модификации и делает затруднительной адаптацию под специфические инженерные или исследовательские задачи. В качестве альтернативы можно рассмотреть использование микроконтроллерных платформ и встроенных систем с поддержкой сетевых интерфейсов, благодаря чему появится возможность самостоятельно сформировать аппаратно-программную архитектуру устройства, настроив ее под конкретные специфические требования при относительно низкой стоимости и высокой степени интеграции компонентов.

Чтобы создавать такие устройства на практике обычно используются микроконтроллеры семейства ESP32, у которых есть встроенные модули Wi-Fi [5]. Вместе с этим при разработке таких решений возникает ряд технических проблем, связанных с доступом к файловой системе, правильным переключением между проводным и беспроводным режимами работы, а также реализацией пользовательского интерфейса для управления данными удаленно. Исходя из таких особенностей и ограничений вытекает то, что задача создания автономного USB-накопителя с беспроводным доступом к данным остается нерешенной.

Объект исследования – автономные системы хранения данных с проводным и беспроводным доступом.

Предмет исследования – архитектура и методы реализации USB-накопителя с функцией облачного доступа к данным.

Цель дипломной работы – спроектировать и реализовать USB Wi-Fi накопитель, обеспечивающий локальный доступ к данным через интерфейс USB и удаленный доступ через встроенный Wi-Fi модуль и веб-интерфейс без использования внешних облачных сервисов.

Для достижения поставленной цели в рамках дипломной работы необходимо решить следующие задачи:

- проанализировать предметную область автономных USB-накопителей с беспроводным доступом к данным;
- выполнить обзор существующих технических решений и аналогов;
- спроектировать аппаратно-программную архитектуру Wi-Fi USB-накопителя;
- выбрать программные и аппаратные средства реализации устройства;
- разработать программное обеспечение для работы накопителя в режимах «USB Mass Storage» и Wi-Fi сервера;

- реализовать веб-интерфейс для управления файловым хранилищем устройства;
- провести тестирование разработанного прототипа и оценить его работоспособность.

В процессе выполнения дипломной работы использовались такие методы исследования как: анализ и обобщение научно-технических источников, проектирование аппаратных и программных систем встроженных устройств, тестирование, а также методы моделирования режимов работы и оценки устойчивости функционирования файловой системы.

1 Анализ предметной области и существующих решений

1.1 Автономные USB-накопители и системы хранения данных

USB-накопители по прежнему широко используются для хранения и переноса цифровых данных. На практике это объясняется их простотой в использовании и тем, что такие устройства отлично работают в большинстве распространенных операционных систем без дополнительной настройки. Подключение через стандартный интерфейс USB позволяет использовать накопители с самыми разными устройствами: от персональных компьютеров и ноутбуков до принтеров и встраиваемых систем [6].

Классическое устройство хранения используется как съемный накопитель и предоставляет доступ к файловой системе через интерфейс USB при прямом подключении к хост-устройству. Диск определяется операционными системами как устройство класса «USB Mass Storage». Компьютеры настроены на распознавание этого класса, поэтому для работы флешек не нужно устанавливать дополнительные драйверы [7]. Это очень удобно и привычно пользователю, но при всех этих достоинствах доступ к данным напрямую привязывается к физическому соединению и не предусматривает других способов работы с содержимым накопителя. Такой недостаток так же усложняет получение информации с носителя в условиях, когда у пользователя нету доступа к компьютеру или другому хост-устройству [8]. Помимо этого, архитектура классического USB диска не подразумевает возможность работы с файлами сразу несколькими пользователями.

Из-за увеличения объемов хранимой информации, пространства на обычных накопителях стало недостаточно. Популярными стали другие способы хранения файлов, например, сетевые хранилища или облачные диски, с помощью которых можно работать с данными через сеть [9]. Обычно это упрощает совместную работу, но для этого необходимо стабильное и быстрое подключение к сети, а также доступность сервиса, на котором хранится информация. При нестабильном соединении или в плохих условиях доступ к данным оказывается невозможным. Использование сторонних сервисов подразумевает то, что данные физически находятся у третьих лиц, что не всегда допустимо при работе с важной и конфиденциальной информацией.

Автономные устройства хранения данных позволяют расширить способы получения доступа к информации с USB-накопителей. Это достигается за счет внедрения дополнительных механизмов доступа и управления. Хранилища могут применяться для обмена файлами между несколькими клиентскими устройствами одновременно, обновления данных и обслуживания оборудования. Им нет необходимости для прямого подключения к каждому элементу системы. Для автономных систем важно, чтобы устройство работало самостоятельно, без дополнительного

вмешательства пользователя. Доступ к данным должен сохраняться даже при отсутствии подключения к сети, а само устройство обязано быть устойчивым к сбоям [10]. Исходя из этого, ключевыми требованиями являются: сохранность данных, восстановление после ошибок, минимальные аппаратные и программные затраты, независимость от внешних сервисов и инфраструктуры.

Классические USB-накопители не удовлетворяют требованиям современных автономных и мобильных устройств, даже несмотря на свою простоту и надежность. Это главная причина, по которой разработка гибридных решений необходима и актуальна для расширения возможностей доступа к информации.

1.2 Беспроводной доступ к данным и облачные технологии

Развитие сетевых технологий уверенно расширило способы работы пользователей с цифровой информацией и изменило вид современных систем хранения данных. Самым распространенным и популярным способом беспроводного доступа стало использование технологии Wi-Fi. Она обеспечивает передачу данных в пределах локальной сети без физического подключения устройств. Благодаря обширной поддержке со стороны производителей компонентов, Wi-Fi стал базовым инструментом для создания локальных сетей и беспроводного доступа.

Беспроводной доступ позволяет обращаться к данным без жесткой привязки к физическим интерфейсам, в сравнении с обычным проводным подключением [11]. Это позволяет использовать устройства, когда доступ физически ограничен или невозможен. Например, если их несколько, они находятся далеко, или у них вовсе нет разъемов для проводного подключения. Помимо этого, устройства, использующие технологию Wi-Fi, могут как подключаться к существующим локальным сетям, так и создавать свои для подключения других устройств.

Благодаря успеху в развитии беспроводных технологий, популярность стали набирать облачные системы и сервисы. Они ориентированы на создание удаленного доступа к информации через глобальную сеть [12]. Облачные сервисы удобны тем, что могут обеспечивать синхронизацию файлов с пользовательскими устройствами, но в то же время они требуют наличия стабильного интернет-подключения. Передача данных на такие сервисы может быть опасна, так как конечный пользователь не знает как их информацию использует сервис и его владельцы, если этого не указано в политиках конфиденциальности.

В некоторых практических случаях применение внешних облачных сервисов оказывается избыточным. Например, когда пользователю

необходимо провести обслуживание оборудования или обновить его программное обеспечение. Промышленные станки могут вовсе не поддерживать беспроводное подключение. То же самое происходит если у пользователя нет прямого доступа к глобальной сети. В таких условиях преимущество остается за локальными сетевым решениям или проводными устройствами хранения.

Остается открытым вопрос о том, как получать доступ к файлам на сетевых хранилищах. Обычно производители таких систем реализуют протоколы удаленной передачи файлов, например, FTP или WebDAV [13]. Но чаще всего такие протоколы требуют наличия у пользователя специального программного обеспечения для создания подключения. Это не всегда удобно, к тому же операционная система устройства может не поддерживать установку этих программ. Другим довольно популярным подходом является создание специального веб-интерфейса. Пользователь может использовать любое доступное ему базовое устройство, так как для доступа к данным достаточно воспользоваться стандартным веб-браузером [14].

При проектировании автономных устройств, объединяющих проводной и беспроводной подходы доступа к данным, должно уделяться внимание к вопросам устойчивости системы и сохранности данных. Использование беспроводного интерфейса не должно приводить к конфликтам при одновременном доступе к файловой системе. Переключение режимов работы не должно нарушать целостность файловой системы. Ограничения аппаратных ресурсов микроконтроллеров накладывают дополнительные требования на выбор архитектурных решений и алгоритмов управления данными.

1.3 Анализ существующих решений и аналогов

Современный рынок насыщен различными решениями для обеспечения беспроводного доступа к данным, хранящимся на съемных носителях. К наиболее распространенным относятся Wi-Fi адаптеры для SD-карт памяти, коммерческие USB-накопители с поддержкой Wi-Fi, и автономные файловые серверы [15]. Несмотря на схожесть функционального назначения, эти устройства сильно отличаются друг от друга по архитектуре и способам доступа к данным.

Карты памяти и адаптеры, обеспечивающие Wi-Fi-доступ к данным, востребованы фотографами и видеографами. Представителями таких решений являются «Toshiba FlashAir» и «Eye-Fi Mobi» [16; 17]. Такие накопители используются в фото- и видеотехнике. Они позволяют просмотреть файлы с мобильных устройств, используя специальные приложения и беспроводное подключение. Но у таких устройств есть ряд недостатков. Некоторые из них

поддерживают хранение и просмотр только фотографий определенного формата. Главным недостатком является то, что чтобы получить доступ к файлам, нужно подключиться к Wi-Fi сети устройства, то есть его нельзя подключить к домашней сети.

Коммерческие Wi-Fi USB-накопители, как правило, представляют собой закрытые устройства с предустановленным программным обеспечением и определенным набором функций. Такие устройства позволяют получить доступ к файлам через мобильные приложения или веб-интерфейс, но обладают ограниченными возможностями настройки. В качестве примера ранее существовавших коммерческих решений можно рассмотреть устройство «SanDisk Connect» [18]. Накопитель сочетал функции портативного хранилища и беспроводного сетевого интерфейса. Устройство не отличалось по размеру от современных USB-накопителей. Беспроводной доступ осуществлялся через специальное мобильное приложение. К недостаткам устройства можно отнести то, что беспроводной доступ к нему был невозможен, когда накопитель был подключен к компьютеру или другому хост-устройству. К тому же, для включения функции беспроводного доступа нужно было нажать специальную кнопку на корпусе устройства [19]. Внешний вид накопителя приведен на рисунке 1.3.1.



Рисунок 1.3.1 – Внешний вид устройства «SanDisk Connect»

На текущий момент устройство снято с производства, а мобильное приложение, с помощью которого предоставлялся беспроводной доступ, больше не поддерживается и было удалено из магазинов приложений [20].

На рынке также встречаются устройства, построенные на базе одноплатных компьютеров и микроконтроллерных платформ. Первые позволяют реализовать полнофункциональные сетевые хранилища, однако отличаются более высокими энергопотреблением, габаритами и стоимостью [21]. Это делает их избыточными для задач компактного автономного хранения данных. Микроконтроллерные решения, наоборот, обладают меньшими

аппаратными ресурсами, но позволяют создавать энергоэффективные и компактные устройства, ориентированные на выполнение строго определенных функций. Но такие устройства чаще всего ориентируются на решение только одной задачи: либо организацию доступа к данным через интерфейс USB, либо на беспроводной доступ.

Проведенный анализ показывает, что на сегодняшний день существующие на рынке решения не обеспечивают в полной мере совмещение функций компактного автономного USB-накопителя и сетевого сервера. Это определяет актуальность разработки Wi-Fi USB-накопителя на базе микроконтроллерной платформы.

2 Проектирование и реализация Wi-Fi USB-накопителя

2.1 Постановка задачи и требования к системе

На основании анализа предметной области и существующих решений можно сформулировать задачу на разработку автономного устройства хранения данных. Устройство должно сочетать возможности обычного USB-накопителя и функции беспроводного доступа к информации. Проектируемый Wi-Fi USB-накопитель должен предоставлять удобный доступ к файлам как при физическом подключении к хост-устройству, так и через локальное беспроводное соединение без использования сторонних сервисов и инфраструктуры. Это позволит применить устройство в изолированных или ограниченных по сетевым возможностям условиях.

Основной целью проектирования является создание компактного устройства, которое может функционировать в нескольких режимах работы. При этом оно должно обеспечивать сохранность данных при их использовании через различные интерфейсы [22]. Устройство не должно требовать от пользователя установки специализированного программного обеспечения или дополнительных драйверов.

Для достижения поставленных целей в рамках дипломной работы нужно решить следующие основные задачи:

- разработать архитектуру устройства, объединяющую подсистему хранения данных, USB-интерфейс и беспроводной доступ;
- реализовать режим работы устройства в качестве стандартного USB-накопителя;
- реализовать возможность беспроводного доступа к данным по технологии Wi-Fi;
- реализовать встроенный веб-сервер для управления данными и взаимодействия с файловой системой;
- обеспечить корректное переключение между режимами работы без нарушения целостности файловой системы;
- провести тестирование разработанного устройства в основных режимах работы и оценить полученные результаты.

В процессе проектирования к устройству был предъявлен ряд функциональных требований. Они определяют его основные возможности. Разрабатываемый Wi-Fi USB-накопитель должен поддерживать:

- работу в режиме «USB Mass Storage» при подключении к хост-устройству;
- хранение данных на носителе с файловой системой, которая совместима с операционными системами Windows, Linux и MacOS;
- возможность смены параметров беспроводного подключения и изменения настроек системы;

- удаленный доступ к данным через локальную Wi-Fi сеть;
- простое управление данными через встроенный веб-интерфейс;
- возможность обновления встроенного программного обеспечения в процессе эксплуатации.

К проектируемому устройству также были предъявлены нефункциональные требования. Они направлены на обеспечение надежности и удобства использования. К таким требованиям относятся:

- сохранность данных при смене режимов работы устройства;
- устойчивость к нештатным ситуациям (отключение питания, разрыв соединения);
- минимальные требования к аппаратным ресурсам и энергопотреблению;
- простота использования и настройки устройства пользователем.

В рамках дипломной работы ставится задача разработки Wi-Fi USB-накопителя, представляющего собой автономное устройство хранения данных с поддержкой проводного и беспроводного доступа. Предъявленные требования влияют на выбор архитектурных и программных решений при проектировании устройства.

2.2 Общая архитектура устройства

Разрабатываемый Wi-Fi USB-накопитель представляет собой автономное аппаратно-программное устройство. Архитектура диска построена по модульному принципу. Такая структура позволяет логически разделить функции хранения файлов, проводного и беспроводного доступа к ним, а также управления устройством [23]. При этом обеспечивается простота отладки, устойчивость работы устройства, возможность расширения функций без изменения базовых модулей системы.

Общий вид архитектуры устройства состоит из:

- аппаратной микроконтроллерной платформы;
- подсистемы хранения файлов;
- интерфейса USB для проводного подключения к хост-устройствам;
- беспроводной сетевой подсистемы;
- других программных модулей, связывающих предыдущие.

Все подсистемы работают и взаимодействуют друг с другом в рамках единой встроенной прошивки. и подчиняются главному модулю устройства. Работа между подсистемами устройства организована так, чтобы уменьшить взаимное влияние интерфейсов при обращении к файловой системе. Это значит, что архитектура устройства предполагает согласованное и очередное управление доступом к данным. Такой режим работы помогает избежать конфликтов при переключении между режимами проводного и беспроводного

доступа. Это также упрощает реализацию логики устройства, уменьшает число ошибок и повышает надежность и отказоустойчивость устройства [24]. Общая архитектура устройства и взаимосвязь его основных подсистем представлены на рисунке 2.2.1.



Рисунок 2.2.1 – Структурная схема устройства

Главным модулем архитектуры является микроконтроллер и его встроенные возможности, который выполняет функции управления аппаратными ресурсами микроконтроллера, обработки сетевых обращений и организации доступа к файловой системе. К микроконтроллеру подключается съемный диск. Он используется для хранения пользовательских файлов. Доступ к данным производится через файловую систему FAT32, которая совместима с распространенными операционными системами, например: Windows, Linux и MacOS [25].

Архитектура устройства предусматривает поддержку двух режимов работы: проводной режим и режим встроенного веб-сервера. Они определяют способ работы пользователя с файлами. В режиме проводного подключения устройство определяется как стандартный USB-накопитель и дает хост-устройству прямой доступ к файловой системе. В режиме веб-сервера обеспечивается беспроводной доступ к данным по технологии Wi-Fi. В этом случае накопитель выступает в роли автономного сетевого хранилища. Переключение между режимами осуществляется программно и учитывает контроль состояния файловой системы для исключения конфликтов доступа и потери данных.

Для создания беспроводного доступа в архитектуре устройства предусмотрена сетевая подсистема, которая инициализирует Wi-Fi модуль и обрабатывает сетевые запросы. Веб-сервер дает пользователю доступ к данным с помощью протокола HTTP [26]. Использование веб-ориентированного подхода позволяет пользователю работать с файлами с помощью обычного веб-браузера без установки дополнительного программного обеспечения.

Одним из наиболее важных элементов архитектуры является подсистема управления конфигурацией устройства. Модуль отвечает за хранение и обработку настроек устройства. Конфигурационные данные считываются при старте устройства. Остальные подсистемы используют полученную информацию для настройки отдельных параметров.

Общая архитектура Wi-Fi USB-накопителя представляет собой совокупность взаимосвязанных аппаратных и программных компонентов, обеспечивающих хранение данных и доступ к ним через проводной и беспроводной интерфейсы. Принцип модульности и четкое разделение функций между подсистемами формируют основу надежной работы устройства и служат базой для дальнейших этапов проектирования и реализации.

2.3 Аппаратная реализация устройства

Аппаратная реализация разрабатываемого Wi-Fi USB-накопителя основана на использовании микроконтроллерной платформы, обеспечивающей аппаратную поддержку интерфейсов USB и Wi-Fi, а также достаточные вычислительные ресурсы для управления файловой системой и обработки сетевых запросов. В качестве аппаратной основы устройства был выбран микроконтроллер ESP32-S3 в исполнении «Super Mini», который сочетает в себе компактные размеры, встроенный радиомодуль Wi-Fi и нативную поддержку режима «USB OTG» [27]. Это позволяет решить поставленные задачи без применения дополнительных внешних контроллеров и модулей [28]. Внешний вид используемого микроконтроллера приведен на рисунке 2.3.1.

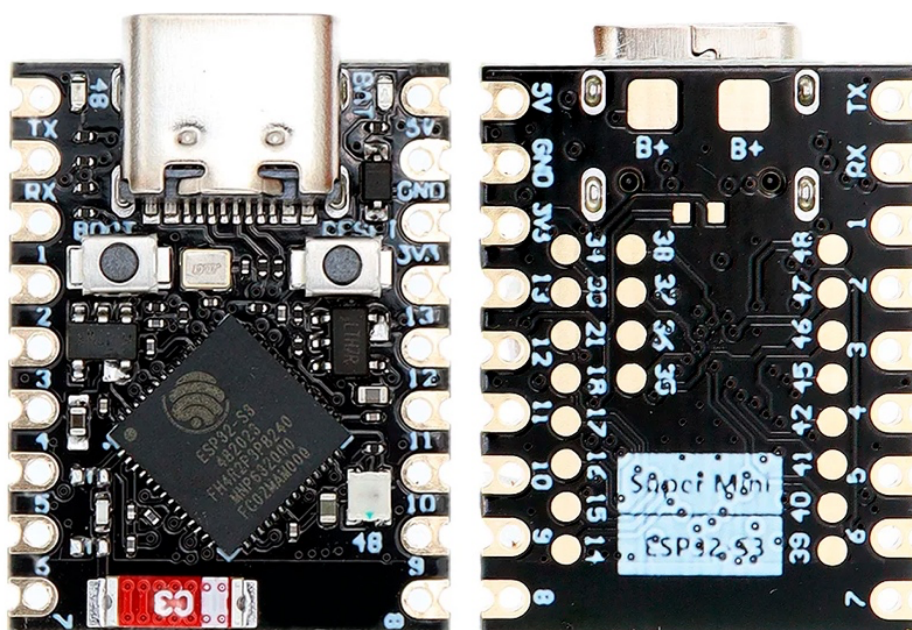


Рисунок 2.3.1 – Микроконтроллер ESP32-S3 «Super Mini»

Выбор микроконтроллера ESP32-S3 обусловлен его архитектурными возможностями, важными для создания автономного устройства хранения данных. Платформа поддерживает аппаратную реализацию USB-интерфейса, что позволяет использовать устройство в режиме «USB Mass Storage» и обеспечивать совместимость с распространенными операционными системами. Наличие встроенного Wi-Fi-модуля упрощает аппаратную часть устройства за счет нативной обработки операций в ядре процессора и снижает энергопотребление по сравнению с решениями, использующими внешние сетевые контроллеры [29]. Выбранный микроконтроллер оснащен двухъядерным процессором, 512 килобайтами оперативной памяти и 4 мегабайтами встроенной флэш-памяти [30]. Такие ресурсы достаточны для одновременной работы сетевых сервисов и управления файловой системой.

В качестве носителя пользовательских файлов в устройстве используется съемная карта памяти формата microSD. Подключение карты памяти к микроконтроллеру осуществляется по интерфейсу SDMMC, что позволяет обеспечить более высокую пропускную способность по сравнению с альтернативным вариантом подключения: SPI [31; 32]. Использование интерфейса SDMMC особенно важно для задач передачи файлов большого объема и обеспечивает приемлемую производительность как в режиме USB-доступа, так и при беспроводном взаимодействии. Применение съемного носителя также упрощает обслуживание устройства и позволяет гибко выбирать максимальный объем хранимых данных.

Интерфейс USB реализуется с использованием встроенных аппаратных возможностей микроконтроллера ESP32-S3 и предназначен для подключения устройства к персональным компьютерам и другим хост-устройствам. При подключении по USB устройство определяется системой как стандартный накопитель, что обеспечивает его прозрачную интеграцию в операционную систему. Аппаратная реализация USB-интерфейса учитывает требования по питанию разъема и самого устройства, инициализации и корректному завершению работы при отключении устройства от хоста.

Беспроводное взаимодействие обеспечивается с использованием встроенного Wi-Fi-модуля микроконтроллера. Аппаратная часть Wi-Fi-подсистемы не требует применения дополнительных внешних компонентов, что положительно сказывается на компактности и надежности конструкции. Параметры работы радиомодуля и антенного тракта определяются с помощью программных настроек и соответствуют требованиям для работы в локальных сетях стандарта IEEE 802.11 [33].

Питание устройства осуществляется через интерфейс USB, что упрощает эксплуатацию и исключает необходимость внедрения в схему аккумуляторов. Аппаратная схема обеспечивает стабильное питание

микроконтроллера и карты памяти, одинаковое для различных режимов работы. При проектировании учитывались ограничения по потребляемому току, а также особенности работы устройства как в режиме USB-подключения, так и при автономной эксплуатации с использованием беспроводного интерфейса.

Аппаратная реализация Wi-Fi USB-накопителя представляет собой компактную и архитектурно завершенную микроконтроллерную систему, обеспечивающую поддержку всех необходимых интерфейсов для хранения и передачи данных. Выбранная аппаратная конфигурация формирует надежную основу для реализации программной архитектуры устройства, а также сетевых и сервисных механизмов, используемых в системе.

2.4 Архитектура встроенного программного обеспечения

Программное обеспечение разрабатываемого Wi-Fi USB-накопителя представляет собой встроенную прошивку, выполняющуюся на микроконтроллерной платформе и обеспечивающую согласованную работу аппаратных подсистем устройства. Архитектура программного обеспечения, как и аппаратная архитектура, построена по модульному принципу, что позволяет разделить функциональные обязанности между отдельными компонентами, повысить надежность работы системы и упростить сопровождение и расширение функциональности устройства [34].

В качестве программной платформы для разработки встроенного программного обеспечения использован язык программирования C и официальный фреймворк ESP-IDF, предоставляемый производителем микроконтроллеров ESP32 [35]. Он включает в себя кодовую базу ядер микроконтроллеров различных серий, набор библиотек и инструментов для работы с аппаратными ресурсами устройств, реализации сетевых протоколов и схемы взаимодействия с периферийными устройствами. Использование ESP-IDF обеспечивает доступ к низкоуровневым возможностям платформы, что позволяет получить совместимость с необходимыми аппаратными модулями. Кроме того, применение ESP-IDF позволяет использовать готовые программные компоненты для реализации USB-интерфейса и беспроводного доступа, что снижает объем низкоуровневой разработки, ускоряет построение логических блоков и повышает надежность программного обеспечения.

Управляющая логика программного обеспечения реализована в точке входа системы и отвечает за последовательную инициализацию всех составляющих устройства. В процессе запуска выполняется подготовка аппаратных ресурсов, загрузка конфигурационных параметров, инициализация беспроводного соединения, настройка подсистемы хранения данных и запуск встроенных веб-сервисов. Последовательный порядок

инициализации позволяет обеспечить корректную работу устройства в различных режимах эксплуатации и исключить конфликтное использование общих ресурсов.

Одним из ключевых элементов программной архитектуры является подсистема управления доступом к съемной памяти. Этот модуль отвечает за инициализацию режима «USB Mass Storage», а также за организацию доступа к данным при использовании встроенного сетевого сервиса. В рамках подсистемы реализована логика переключения между проводным и беспроводным режимами работы устройства. Это обеспечивает корректное использование файловой системы и предотвращает повреждение данных при изменении режима работы.

Для обеспечения целостности файловой системы и предотвращения одновременного конфликтующего доступа к файлам в программной архитектуре предусмотрен модуль контроля активности интерфейсов. Он отслеживает текущее использование USB и веб-интерфейсов и блокирует недопустимые операции переключения режимов в случае активного использования съемного носителя. Такой подход позволяет исключить ситуации одновременного доступа к файловой системе со стороны различных интерфейсов и повысить отказоустойчивость устройства [36].

Сетевая подсистема программного обеспечения отвечает за инициализацию и поддержку беспроводного соединения по технологии Wi-Fi. В рамках этой подсистемы производится считывание сетевых параметров из конфигурационного файла, настройка режимов работы беспроводного интерфейса и поддержание сетевого соединения в процессе эксплуатации устройства. Отделение сетевой логики в отдельный модуль упрощает настройку устройства и позволяет изменять параметры подключения без внесения изменений в другие модули и подсистемы.

Поверх сетевой подсистемы реализован встроенный сетевой веб-сервис. Он обеспечивает работу пользователя с устройством через стандартный веб-браузер. Данный сервис отвечает за обработку HTTP-запросов, маршрутизацию команд и выполнение операций с файловой системой устройства. Использование веб-ориентированного подхода позволяет обеспечить доступ к данным без установки специализированных программ и приложений и делает процесс взаимодействия с устройством интуитивно понятным [37].

Для расширения функциональных возможностей при работе с файлами в программной архитектуре используются вспомогательные программные компоненты. В частности, реализована поддержка упаковки и распаковки архивов, применяемая при загрузке и скачивании каталогов через веб-интерфейс. Использование вспомогательной библиотеки обработки архивов

позволяет выполнять такие операции непосредственно на устройстве без привлечения сторонних сервисов.

Особенно важной частью в программной архитектуре является подсистема обновления встроенного программного обеспечения. Этот сервис осуществляет проверку наличия файла с новой версией прошивки на съемном носителе и выполняет процедуру обновления устройства. Механизм обновления используется как сервисная функция и не влияет на основные режимы работы устройства, обеспечивая при этом удобство сопровождения и модернизации системы.

Программная архитектура встроенного программного обеспечения Wi-Fi USB-накопителя представляет собой модульную систему с централизованным управлением и четким разделением функциональных обязанностей между компонентами. Такая архитектура обеспечивает надежную работу устройства, сохранность данных при использовании различных интерфейсов и формирует основу для реализации конкретных функциональных механизмов.

2.5 Реализация режима USB Mass Storage

Одним из основных режимов работы разрабатываемого Wi-Fi USB-накопителя является режим проводного доступа к данным, реализуемый с использованием интерфейса USB. В этом режиме устройство работает как стандартный USB-накопитель и предоставляет хост-устройству прямой доступ к файловой системе носителя данных. Реализация такого режима обеспечивает совместимость устройства со всеми распространенными операционными системами и не требует установки дополнительного программного обеспечения или драйверов на стороне пользователя.

Режим «USB Mass Storage» реализован с использованием встроенных аппаратных возможностей микроконтроллера и компонента «TinyUSB», обеспечивающих эмуляцию стандартного накопителя [38]. Библиотека предоставляет функции низкоуровневого управления интерфейсом USB и служит мостом для монтирования хранилища в систему хост-устройства [39]. При подключении накопителя к хост-системе осуществляется инициализация USB-интерфейса и регистрация устройства как накопителя класса «Mass Storage». В дальнейшем все операции чтения и записи, инициируемые хост-устройством, обрабатываются микроконтроллером при помощи низкоуровневых функций компонента и транслируются в обращения к файловой системе съемного носителя. Пример программной инициализации режима «USB Mass Storage» приведен в листинге 2.5.1.

Листинг 2.5.1 – Инициализация проводного режима

```
void msc_init(void)
```

```

{
    ESP_LOGI(TAG, "MSC init started");
    esp_err_t ret = storage_sdmmc_init();
    if (ret != ESP_OK) {
        ESP_LOGE(TAG, "MSC init failed: SDMMC error");
        free(_sd_card);
        return;
    }
    const tinyusb_msc_storage_config_t config_sdmmc = {
        .mount_point = TINYUSB_MSC_STORAGE_MOUNT_USB,
        .fat_fs = {
            .base_path = BASE_PATH,
            .config = _mount_config,
            .format_flags = 0, },
        .medium.card = _sd_card, };
    ret = tinyusb_msc_new_storage_sdmmc(&config_sdmmc,
    &_msc_handle);
    if (ret != ESP_OK) {
        ESP_LOGE(TAG, "TinyUSB MSC init failed: %s",
    esp_err_to_name(ret));
        free(_sd_card);
        return;
    }
    tinyusb_config_t tusb_cfg = TINYUSB_DEFAULT_CONFIG();
    tusb_cfg.descriptor.device = &descriptor_config;
    tusb_cfg.descriptor.full_speed_config =
    _msc_fs_configuration_desc;
    tusb_cfg.descriptor.string = _string_desc_arr;
    tusb_cfg.descriptor.string_count = sizeof(_string_desc_arr)
    / sizeof(_string_desc_arr[0]);
    ret = tinyusb_driver_install(&tusb_cfg);
    if (ret != ESP_OK) {
        ESP_LOGE(TAG, "TinyUSB driver install failed: %s",
    esp_err_to_name(ret));
        tinyusb_msc_delete_storage(_msc_handle);
        free(_sd_card);
        return;
    }
    is_usb_disconnected = false;
    ESP_LOGI(TAG, "MSC initialized");
}

```

Подсистема USB-доступа тесно интегрирована с подсистемой управления файлами и использует ее для выполнения операций чтения и записи. Для обеспечения корректной работы файловой системы при использовании проводного доступа предусмотрена централизованная логика управления режимами работы устройства. Это позволяет гарантировать, что в проводном режиме доступ к данным осуществляется исключительно со стороны хост-устройства.

Особое внимание при реализации проводного режима работы уделено предотвращению конфликтов доступа к файловой системе. В случае активного использования USB-интерфейса программное обеспечение микроконтроллера блокирует возможность одновременного обращения к данным через встроенный сетевой сервис. Контроль состояния интерфейсов осуществляется с использованием отдельного программного модуля, отслеживающего активность проводного и беспроводного режимов работы и запрещающего недопустимые операции переключения. Пример проверки состояния интерфейса USB приведен в листинге 2.5.2.

Листинг 2.5.2 – Проверка состояния интерфейса USB

```
static void usb_monitor_task(void *arg)
{
    int64_t usb_timeout = 5 * 1000 * 1000;
    int64_t current_time = 0;
    while (true)
    {
        if (!is_usb_disconnected)
        {
            current_time = esp_timer_get_time();
            ESP_LOGI(TAG, "Time after last usb use: %lld",
(current_time - last_usb_request_time) / 1000 / 1000);
            if (current_time - last_usb_request_time >=
usb_timeout)
            {
                is_usb_busy = false;
                ESP_LOGI(TAG, "USB is already not busy");
            }
            else
            {
                is_usb_busy = true;
            }
        }
        vTaskDelay(pdMS_TO_TICKS(100));
    }
}
```

Микроконтроллеры семейства ESP32 используют операционную систему реального времени «FreeRTOS». Она используется для реализации многозадачности, в том числе для одноядерных процессоров, где каждая задача работает независимо от других [40; 41]. Подсистема контроля активности интерфейсов так же построена на этом принципе. Такой подход удобен для выполнения фоновых задач, не влияющих на основной цикл выполнения программы.

Переключение устройства в проводной режим осуществляется программно и сопровождается предварительной подготовкой файловой подсистемы. В частности, выполняется проверка текущего состояния и

корректное завершение сетевых операций, если устройство ранее использовалось в беспроводном режиме. Важно правильно перемонтировать файловую систему, чтобы исключить возможность потери данных и обеспечить устойчивую работу устройства при смене режима использования.

Реализованный режим «USB Mass Storage» обеспечивает прозрачную интеграцию устройства в операционную систему хост-устройства. Накопитель определяется системой как стандартное внешнее устройство хранения данных благодаря полноценным и правильным дескрипторам устройства. Это позволяет выполнять операции копирования, удаления и изменения файлов с использованием штатных средств операционных систем.

Создание такого режима управления хранилищем обеспечивает надежный и совместимый проводной доступ к данным, хранящимся в файловой системе. Использование централизованного управления режимами работы и механизма контроля активности интерфейсов позволяет сохранить целостность файловой системы и обеспечить корректную работу устройства при использовании разных способов доступа к данным.

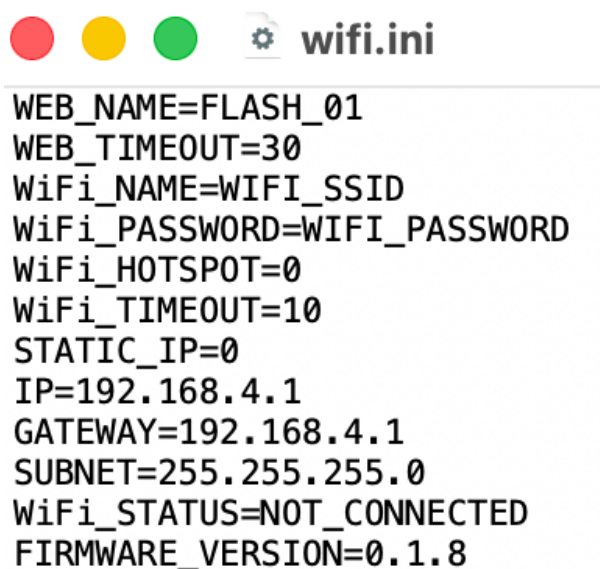
2.6 Реализация беспроводного доступа по Wi-Fi и управления конфигурацией

Беспроводной режим доступа в разрабатываемом Wi-Fi USB-накопителе реализуется с использованием технологии Wi-Fi. Он предназначен для взаимодействия пользователя с устройством без необходимости физического подключения к хост-системе. Этот режим работы позволяет обращаться к файлам, хранящимся на съемном носителе, через локальную беспроводную сеть и обеспечивает автономность эксплуатации накопителя.

Реализация беспроводного доступа организована на основе выделенной сетевой подсистемы программного обеспечения, отвечающей за инициализацию и поддержку Wi-Fi-соединения [42]. Настройка параметров беспроводного подключения производится с использованием файла настроек, хранящегося на подключенной флэш-памяти. Это решение позволяет изменять параметры сети без необходимости перепрошивки устройства и обеспечивает гибкость его настройки в различных условиях эксплуатации.

Конфигурационные параметры беспроводного подключения считываются из файла конфигурации при запуске устройства. В файле задаются основные параметры сети, включая идентификатор беспроводной сети, ее пароль, а также другие параметры подключения. Загрузка конфигурации выполняется на раннем этапе инициализации системы, что позволяет корректно настроить сетевую подсистему до запуска сервисов, использующих беспроводное соединение. В случае отсутствия или некорректности конфигурационного файла программное обеспечение

устройства использует параметры по умолчанию, обеспечивающие базовую работоспособность беспроводного интерфейса. Формат конфигурационного файла представляет собой текстовый файл. Пример конфигурационного файла приведен на рисунке 2.6.1.



```
WEB_NAME=FLASH_01
WEB_TIMEOUT=30
WiFi_NAME=WIFI_SSID
WiFi_PASSWORD=WIFI_PASSWORD
WiFi_HOTSPOT=0
WiFi_TIMEOUT=10
STATIC_IP=0
IP=192.168.4.1
GATEWAY=192.168.4.1
SUBNET=255.255.255.0
WiFi_STATUS=NOT_CONNECTED
FIRMWARE_VERSION=0.1.8
```

Рисунок 2.6.1 – Конфигурационный файл устройства

Пример считывания параметров беспроводного подключения из конфигурационного файла приведен в листинге 2.6.1.

Листинг 2.6.1 – Считывание параметров конфигурационного файла

```
esp_err_t read_wifi_config(void)
{
    FILE *f = fopen("/data/wifi.ini", "r");
    if (f == NULL)
    {
        fclose(f);
        return ESP_OK;
    }
    char line[128];
    while (fgets(line, sizeof(line), f))
    {
        char *key = strtok(line, "=");
        char *value = strtok(NULL, "\n");
        if (key && value)
        {
            if (strcmp(key, "WEB_NAME") == 0)
                strncpy(wifi_settings.web_name, value,
                    sizeof(wifi_settings.web_name));
            else if (strcmp(key, "WEB_TIMEOUT") == 0)
                wifi_settings.web_timeout = atol(value);
            else if (strcmp(key, "WiFi_NAME") == 0)
                strncpy(wifi_settings.wifi_name, value,
                    sizeof(wifi_settings.wifi_name));
```



```

        else if (strcmp(key, "WiFi_PASSWORD") == 0)
strncpy(wifi_settings.wifi_password, value,
sizeof(wifi_settings.wifi_password));
        else if (strcmp(key, "WiFi_HOTSPOT") == 0)
wifi_settings.wifi_hotspot = atoi(value);
        else if (strcmp(key, "WiFi_TIMEOUT") == 0)
wifi_settings.wifi_timeout = atol(value);
        else if (strcmp(key, "STATIC_IP") == 0)
wifi_settings.static_ip = atoi(value);
        else if (strcmp(key, "IP") == 0)
strncpy(wifi_settings.ip, value, sizeof(wifi_settings.ip));
        else if (strcmp(key, "GATEWAY") == 0)
strncpy(wifi_settings.gateway, value,
sizeof(wifi_settings.gateway));
        else if (strcmp(key, "SUBNET") == 0)
strncpy(wifi_settings.subnet, value,
sizeof(wifi_settings.subnet));
    }
}
fclose(f);
return ESP_OK;
}

```

В рамках реализованного механизма конфигурации предусмотрена возможность задания режима работы беспроводного интерфейса. Устройство может функционировать как в режиме подключения к существующей беспроводной сети, так и в режиме точки доступа, что расширяет сценарии его применения и упрощает первоначальную настройку. Выбор режима работы выполняется на основании параметра «WiFi_HOTSPOT», где значение «0» – это работа в качестве станции, то есть подключение к существующей сети, а «1» – создание собственной точки доступа с заданными именем сети и паролем.

При работе в режиме подключения к существующей сети также предусмотрена возможность задания параметров сетевой адресации, включая использование статического IP-адреса. Это позволяет интегрировать устройство в локальные сети с фиксированной схемой адресации и обеспечивает предсказуемость сетевого взаимодействия при эксплуатации. Для задания статического IP необходимо установить значение «1» в параметр «STATIC_IP», а так же указать IP-адрес, шлюз и маску подсети в соответствующих полях.

Еще одной важной частью файла конфигурации является параметр «WEB_NAME». Он задает имя устройства в сети, которое сопоставляется с IP-адресом с помощью элемента ESP-IDF mDNS. Компонент используется для назначения имени устройству в локальных сетях, у которых нет DNS-сервера,

или он не настроен [43]. Пример инициализации сервиса представлен в листинге 2.6.2.

Листинг 2.6.2 – Инициализация компонента mDNS

```
static void start_mdns_service(const char *hostname)
{
    ESP_LOGI(TAG, "start_mdns_service() called");
    esp_err_t err = mdns_init();
    if (err)
    {
        ESP_LOGE(TAG, "mDNS init failed");
        return;
    }
    mdns_hostname_set(hostname);
    mdns_service_add("CloudCard-WebServer", "_http", "_tcp", 80,
NULL, 0);
}
```

После загрузки конфигурационных данных выполняется инициализация Wi-Fi-подсистемы и установка беспроводного соединения в соответствии с указанными в файле параметрами. В процессе работы устройства сетевая подсистема обеспечивает поддержку соединения и обработку сетевых событий, необходимых для работы встроенных сервисов. Реализация сетевой логики в отдельном программном модуле позволяет изолировать задачи беспроводного взаимодействия от других компонентов системы и упростить сопровождение программного обеспечения. Пример инициализации Wi-Fi-подсистемы с учетом заданного режима работы приведен в листинге 2.6.3.

Листинг 2.6.3 – Инициализация Wi-Fi-подсистемы

```
void start_wifi(void)
{
    ESP_LOGI(TAG, "start_wifi() called");
    esp_err_t ret = nvs_flash_init();
    if (ret == ESP_ERR_NVS_NO_FREE_PAGES || ret ==
ESP_ERR_NVS_NEW_VERSION_FOUND)
    {
        ESP_ERROR_CHECK(nvs_flash_erase());
        ret = nvs_flash_init();
    }
    esp_netif_init();
    esp_event_loop_create_default();
    start_mdns_service(wifi_settings.web_name);
    wifi_start_time = esp_timer_get_time();
    wifi_init_config_t cfg = WIFI_INIT_CONFIG_DEFAULT();
    esp_wifi_init(&cfg);
    if (wifi_settings.wifi_hotspot)
    {
        _wifi_init_ap_mode();
    }
    else
    {

```

```

        _wifi_init_sta_mode();
    }
    esp_wifi_set_max_tx_power(80);
}

```

Беспроводной режим работы устройства интегрирован с подсистемой управления доступом к файловой системе и предполагает контролируемое переключение режимов работы. Для предотвращения конфликтов доступа к файловой системе программное обеспечение устройства не допускает одновременное использование проводного и беспроводного интерфейсов. При активной работе встроенного веб-интерфейса переключение устройства в режим USB осуществляется по инициативе пользователя, либо автоматически, после истечения заданного таймаута при отсутствии сетевой активности со стороны пользователя. Такой алгоритм обеспечивает корректное завершение текущих сетевых операций и способствует сохранению целостности данных и надежной работе файловой системы.

Использование конфигурационного файла для задания параметров беспроводного подключения, а также модульная организация сетевой подсистемы позволяют реализовать беспроводной доступ к данным без увеличения сложности пользовательского взаимодействия. Пользователю не требуется установка дополнительного программного обеспечения или выполнение сложных процедур настройки, что делает процесс использования устройства интуитивно понятным. Реализованный механизм беспроводного доступа по Wi-Fi обеспечивает автономную работу устройства. Интеграция сетевой подсистемы с механизмами управления доступом к файловой системе позволяет обеспечить корректную и безопасную работу устройства при использовании беспроводного интерфейса.

2.7 Реализация встроенного веб-сервера и пользовательского интерфейса

Для обеспечения беспроводного взаимодействия пользователя с устройством и файловой системой реализован встроенный веб-сервер. Модуль предназначен для предоставления доступа к данным, хранящимся на съемном носителе, а также для выполнения управляющих операций посредством веб-страницы. Использование веб-ориентированного интерфейса позволяет отказаться от установки специализированных клиентских программ и обеспечивает универсальность доступа с различных устройств.

Веб-сервер работает в рамках беспроводного режима работы устройства и использует сетевую подсистему, инициализированную на предыдущем этапе. Обработка запросов осуществляется полностью на стороне устройства и включает маршрутизацию входящих HTTP-запросов, выполнение

соответствующих маршрутам операций и формирование ответов для клиента [44]. Архитектура веб-сервера построена таким образом, чтобы изолировать логику сетевого взаимодействия от подсистемы хранения данных и механизмов управления режимами работы устройства. Пример регистрации маршрутов приведен в листинге 2.7.1.

Листинг 2.7.1 – Регистрация маршрутов встроенного веб-сервера

```
void start_webserver(void)
{
    httpd_handle_t server = NULL;
    httpd_config_t config = HTTPD_DEFAULT_CONFIG();
    config.stack_size = 16384;
    if (httpd_start(&server, &config) == ESP_OK)
    {
        httpd_register_uri_handler(server, &_root);
        httpd_register_uri_handler(server,
        &_api_check_device_state);
        httpd_register_uri_handler(server,
        &_api_get_device_name);
        httpd_register_uri_handler(server,
        &_api_get_device_storage_space);
        httpd_register_uri_handler(server, &_api_files);
        httpd_register_uri_handler(server, &_api_create_folder);
        httpd_register_uri_handler(server, &_api_switch_mode);
        httpd_register_uri_handler(server, &_api_force_switch);
        httpd_register_uri_handler(server, &_api_clear);
        httpd_register_uri_handler(server, &_api_lock);
        httpd_register_uri_handler(server, &_api_delete);
        httpd_register_uri_handler(server, &_api_view);
        httpd_register_uri_handler(server, &_api_download);
        httpd_register_uri_handler(server, &_api_upload);
        httpd_register_uri_handler(server, &_api_rename);
        httpd_register_uri_handler(server, &_api_heartbeat);
    }
}
```

Пользовательский интерфейс реализован в виде веб-страницы, доступной через веб-браузер клиентского устройства. Важно, чтобы пользователь был подключен к сети устройства, если оно настроено как точка доступа, или к локальной сети, если устройство работает как станция. Через этот интерфейс пользователь может выполнять операции навигации по файловой системе, управления файлами и каталогами, а также загружать и скачивать файлы и папки. Кроме того, веб-интерфейс предоставляет средства управления режимами работы устройства и выполнения сервисных операций, например, полную очистку накопителя или блокировку файлов и папок. Такой подход обеспечивает интуитивно понятное взаимодействие с устройством и минимизирует требования к подготовке пользователя. Внешний вид

пользовательского веб-интерфейса управления устройством представлен на рисунке 2.7.1.

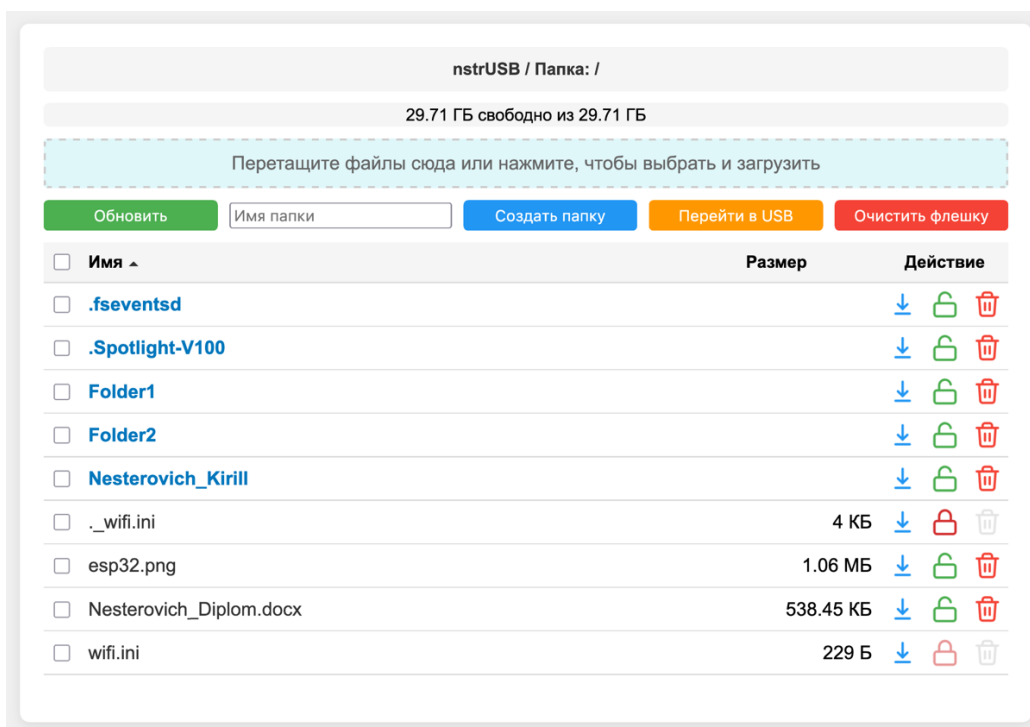


Рисунок 2.7.1 — Веб-интерфейс управления устройством

Для повышения удобства работы с файловой системой в веб-интерфейсе предусмотрена поддержка операций с каталогами. В частности, при скачивании каталогов используется механизм упаковки данных в архив при помощи библиотеки «miniz», что позволяет передавать группу файлов в виде единого объекта [45]. Аналогичным образом реализована возможность загрузки архивов с последующей распаковкой их содержимого на носителе устройства. Данные операции выполняются непосредственно на стороне устройства и не требуют использования внешних сервисов.

Еще одной удобной функцией работы обработки данных является механизм блокировки файлов. Он создан для предотвращения случайного удаления важного файла или каталога пользователем при массовой очистке накопителя, например, при нажатии кнопки «Очистить флешку». Алгоритм построен на основании считывания и редактирования метаданных. В частности, для блокировки используется атрибут «ReadOnly». При этом файл конфигурации «wifi.ini» заблокирован без возможности разблокировки, с целью сохранения пользовательских настроек. Пример смены состояния блокировки файлов представлен в листинге 2.7.2.

Листинг 2.7.2 – Метод смены состояния блокировки файлов

```
static esp_err_t set_file_lock_state(const char *path, uint8_t
state) { // вспомогательный метод для переключения состояния
блокировки файла
```

```

    if (strstr(path, "wifi.ini") != NULL) {
        return ESP_OK;
    }
    FILINFO fno;
    FRESULT res = f_stat(path + 6, &fno);
    if (res != FR_OK) {
        return ESP_ERR_NOT_FOUND;
    }
    res = f_chmod(path + 6, state ? AM_RDO : 0, AM_RDO);
    if (res != FR_OK) {
        return ESP_FAIL;
    }
    return ESP_OK;
}

```

Пользовательский интерфейс веб-сервера реализован с использованием клиентской логики на языке JavaScript и таблиц стилей CSS, что позволяет обеспечить динамическое обновление содержимого страницы без ее перезагрузки. Взаимодействие с устройством осуществляется посредством асинхронных HTTP-запросов, направляемых встроенному веб-серверу, который выполняет соответствующие операции и возвращает результаты на клиентскую сторону [46]. Такое решение повышает отзывчивость интерфейса и обеспечивает удобство работы с устройством при выполнении операций управления и доступа к данным.

Взаимодействие между встроенным веб-сервером устройства и клиентским браузером организовано с использованием обмена данными в формате JSON [47]. Использование такого формата позволяет передавать структурированные данные в компактном и человекочитаемом виде, что упрощает отладку и анализ сетевого взаимодействия. Информация, переданная веб-серверу таким способом, не требует сложной обработки. Это особенно важно ввиду ограниченных вычислительных ресурсов устройства. Применение JSON позволяет расширять веб-интерфейс, добавляя новые команды и параметры без изменения базовой логики обмена информацией между клиентом и устройством.

Для снижения нагрузки на веб-сервер, некоторые процедуры выполняются на клиентской стороне. Примером такой функции является транслитерация имен файлов и директорий. Она нужна для корректной обработки данных файловой системой и их отображения на веб-странице без дополнительного кодирования русских символов. Процедура вызывается при загрузке файлов или при создании нового каталога посредством веб-страницы, после чего веб-сервер получает от клиентской стороны уже стандартизированное имя. Реализация метода транслитерации имен представлена в листинге 2.7.3.

Листинг 2.7.3 – Метод транслитерации имен

```
function transliterate(text)
{
    text = text.replace(/[ъь]/gi, '');
    const translitMap =
    {
        'а': 'a', 'б': 'b', 'в': 'v', 'г': 'g', 'д': 'd', 'е':
'e', 'ё': 'yo', 'ж': 'zh', 'з': 'z', 'и': 'i', 'й': 'y', 'к':
'k', 'л': 'l', 'м': 'm', 'н': 'n', 'о': 'o', 'п': 'p', 'р': 'r',
'с': 's', 'т': 't', 'у': 'u', 'ф': 'f', 'х': 'h', 'ц': 'c', 'ч':
'ch', 'ш': 'sh', 'щ': 'sch', 'ъ': '', 'ы': 'y', 'ь': '', 'э':
'e', 'ю': 'yu', 'я': 'ya',
        'А': 'A', 'Б': 'B', 'В': 'V', 'Г': 'G', 'Д': 'D', 'Е':
'E', 'Ё': 'Yo', 'Ж': 'Zh', 'З': 'Z', 'И': 'I', 'Й': 'Y', 'К':
'K', 'Л': 'L', 'М': 'M', 'Н': 'N', 'О': 'O', 'П': 'P', 'Р': 'R',
'С': 'S', 'Т': 'T', 'У': 'U', 'Ф': 'F', 'Х': 'H', 'Ц': 'C', 'Ч':
'Ch', 'Ш': 'Sh', 'Щ': 'Sch', 'Ъ': '', 'Ы': 'Y', 'Ь': '', 'Э':
'E', 'Ю': 'Yu', 'Я': 'Ya', ' ': '_'
    };
    return text.split('').map(char => translitMap[char] ||
char).join('');
}
```

Веб-сервер тесно интегрирован с подсистемой управления доступом к носителю данных и механизмами контроля активности интерфейсов. При активном использовании веб-интерфейса устройство функционирует в беспроводном режиме, а переход к проводному режиму осуществляется только после корректного завершения текущих операций либо по инициативе пользователя. Такая организация взаимодействия обеспечивает согласованную работу веб-сервера и файловой системы и предотвращает конфликтный доступ к данным.

Реализация встроенного веб-сервера и пользовательского интерфейса дополняет функциональность Wi-Fi USB-накопителя и обеспечивает удобный беспроводной доступ к данным в автономном режиме. Использование веб-ориентированного подхода в сочетании с механизмами управления режимами работы устройства позволяет обеспечить надежное и безопасное взаимодействие пользователя с файловой системой.

2.8 Реализация механизма обновления встроенного ПО

В процессе эксплуатации автономных встроенных устройств важным аспектом является возможность обновления их программного обеспечения без необходимости использования специализированных средств разработки или прямого подключения к отладочной среде. В рамках разрабатываемого Wi-Fi USB-накопителя реализован механизм обновления прошивки,

предназначенный для упрощения его обслуживания и расширения функционала.

Механизм обновления встроенного программного обеспечения основан на использовании файла прошивки, получаемого при компиляции исходного кода. Пользователю достаточно скопировать файл на носитель устройства и перезагрузить его, после чего процедура обновления выполняется автоматически. Такой подход позволяет выполнять обновление программного обеспечения с применением стандартных средств доступа к файловой системе, без использования внешних сетевых сервисов или дополнительных программных инструментов.

Проверка наличия файла обновления осуществляется на этапе инициализации устройства. При обнаружении файла прошивки программное обеспечение инициирует процедуру обновления, выполняемую в контролируемом режиме [48]. Это позволяет исключить вмешательство пользователя в процессе обновления и снизить вероятность ошибок, связанных с некорректным выполнением операций. Пример проверки наличия файла обновления приведен в листинге 2.8.1.

Листинг 2.8.1 – Проверка наличия файла обновления

```
void ota_update_from_file(void) {
    ESP_LOGI(TAG, "Обновление");
    const char *path = "/data/firmware.bin";
    FILE* f = fopen(path, "rb"); // Проверяем наличие файла
прошивки
    if (f == NULL)
    {
        ESP_LOGI(TAG, "Файл прошивки не найден");
        return;
    }
}
```

В целях обеспечения безопасности процесса обновления реализован механизм защиты содержимого файла прошивки. Устройство принимает файл обновления в зашифрованном виде и перед началом обновления, он подвергается процедуре дешифрования. Для подготовки файла обновления используется вспомогательная утилита, предназначенная для предварительного шифрования исходного бинарного образа прошивки. Утилита применяется только на этапе формирования файла обновления и не входит в состав встроенного программного обеспечения устройства. Дешифрование файла происходит непосредственно на микроконтроллере, используя его встроенные возможности. Использование криптографической защиты позволяет предотвратить несанкционированную замену программного обеспечения и снизить риск установки модифицированных или поврежденных версий прошивки. Реализация шифрования рассматривается как элемент

обеспечения целостности и надежности обновляемого программного обеспечения.

Процедура обновления встроенного программного обеспечения интегрирована с общей архитектурой устройства и учитывает требования к корректной работе файловой системы и аппаратных интерфейсов. На время выполнения обновления устройство отключает все интерфейсы доступа к файловой системе, что предотвращает конфликтный доступ и обеспечивает корректное завершение процесса. После успешного обновления и перезагрузки использованный файл уничтожается, а устройство переходит в рабочий режим, используя новую версию программного обеспечения. Если на каком-то из этапов обновления произойдет ошибка, обновление не будет установлено.

Реализованный механизм обновления прошивки не требует постоянного сетевого подключения и может использоваться в автономных условиях, что соответствует общей концепции разрабатываемого Wi-Fi USB-накопителя. Возможность обновления встроенного программного обеспечения посредством файла на носителе данных делает устройство более гибким в эксплуатации и позволяет вносить изменения и улучшения в программную часть без изменения аппаратной конфигурации.

3 Оценка функционирования и устойчивости системы

3.1 Методика и условия тестирования устройства

Целью тестирования разработанного Wi-Fi USB-накопителя является проверка корректности функционирования устройства в основных режимах работы, а также оценка устойчивости системы при переключении между ними и в потенциально неблагоприятных сценариях использования. Процесс тестирования направлен на подтверждение того, что реализованные архитектурные решения обеспечивают стабильную работу устройства и сохранность пользовательских данных.

Проверка работоспособности проводилась на прототипе устройства, собранном на базе микроконтроллера ESP32-S3 «Super Mini». В качестве носителя данных использовалась карта памяти «SanDisk Ultra» формата «microSDHC», объемом 32 гигабайта и 10 классом скорости.

Тестирование выполнялось в двух основных режимах работы устройства:

- проводной режим с подключением к хост-устройству;
- беспроводной режим с использованием встроенного веб-интерфейса.

В процессе испытаний использовались персональные компьютеры под управлением операционных систем Windows, MacOS, а также мобильное устройство с системой iOS. Для тестирования веб-интерфейса использовались веб-браузеры «Firefox» и «Google Chrome». Такой выбор позволяет оценить универсальность решения и корректность работы устройства в разных средах.

Методика тестирования включала ручную последовательную проверку отдельных функциональных компонентов устройства. Особое внимание уделялось переключению режимов работы, обработке пользовательских ошибок и возможными конфликтами доступа к файловой системе. Дополнительно проверялась устойчивость работы устройства при многократных циклах подключения и отключения интерфейсов.

Выбранный подход к тестированию позволяет не только подтвердить работоспособность отдельных функций, но и оценить поведение устройства как автономной системы в целом.

3.2 Тестирование проводного режима работы

В рамках тестирования проводного режима работы устройство физически подключалось к компьютерам с помощью USB и использовалось как стандартный съемный накопитель. Основной задачей этого этапа являлась проверка корректного определения устройства операционной системой, стабильности работы файловой системы и замеры скорости обмена данными.

При подключении к компьютеру под управлением операционной системы Windows устройство корректно определяется как внешний USB-

накопитель и отображается в проводнике без установки дополнительных драйверов. В проводнике доступны стандартные операции работы с файлами: копирование, удаление, переименование и создание каталогов. Пример отображения устройства в среде Windows приведен на рисунке 3.2.1.

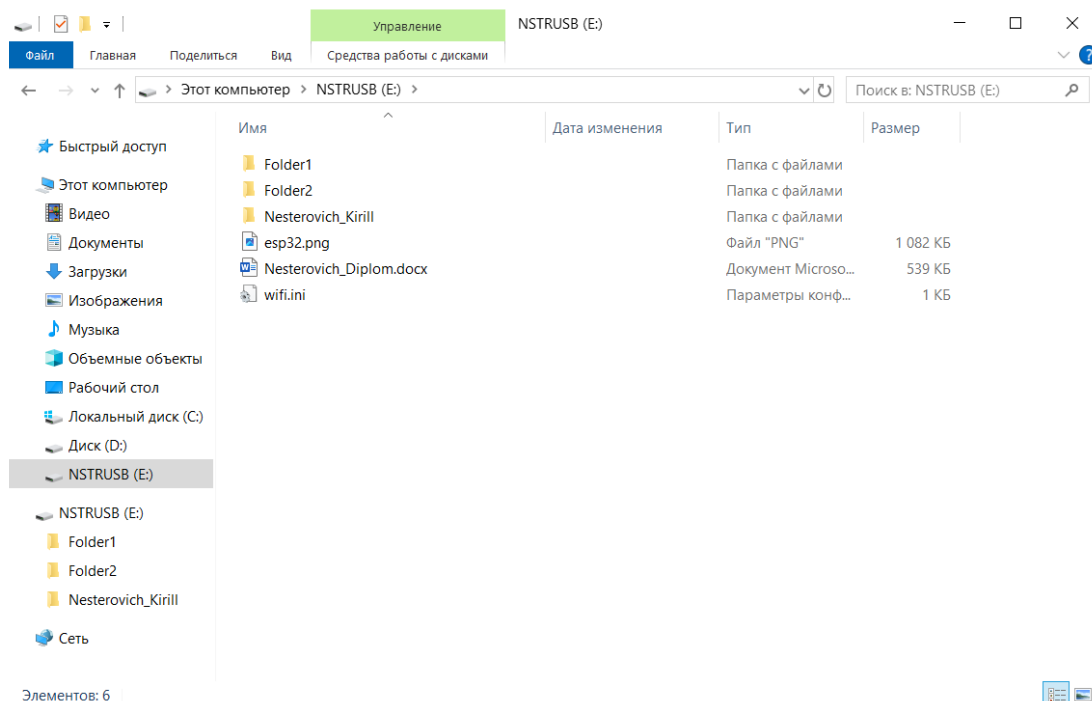


Рисунок 3.2.1 – Отображение устройства в проводнике Windows

Аналогичное поведение зафиксировано при подключении к компьютеру под управлением MacOS. Устройство автоматически монтируется в файловую систему и становится доступным для работы. Пример отображения накопителя в MacOS приведен на рисунке 3.2.2.

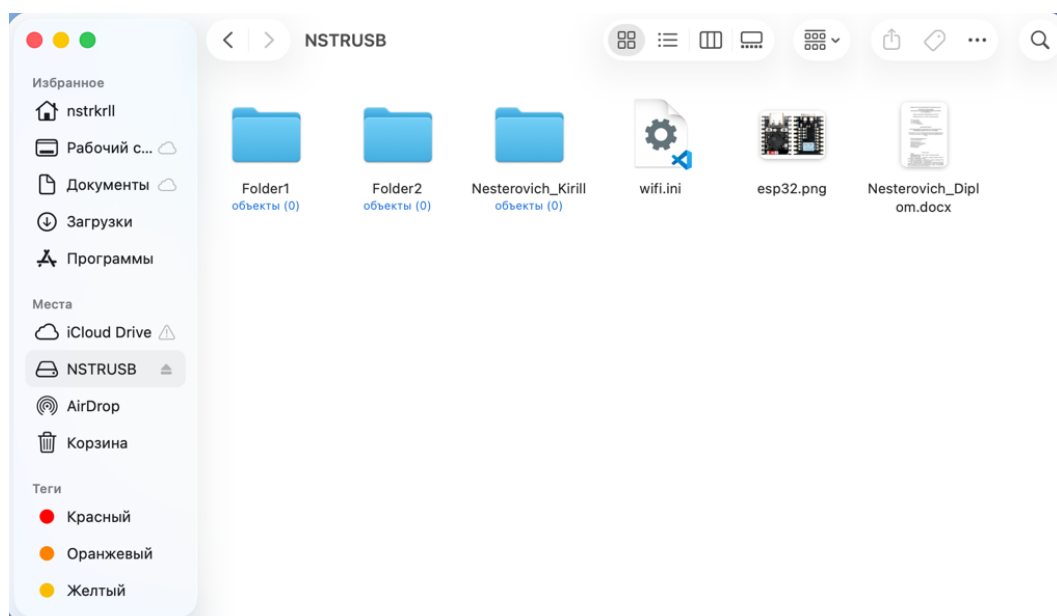


Рисунок 3.2.2 – Отображение устройства в проводнике MacOS

Для оценки производительности проводного режима были выполнены замеры скорости чтения и записи данных. Измерения проводились с помощью утилиты «CrystalDiskMark». Она производит многократные операции чтения и записи файлов разного объема и получает усредненную скорость обмена данными [49]. Полученные результаты приведены на рисунке 3.2.3.

	Read (MB/s)	Write (MB/s)
SEQ1M Q8T1	0.84	0.84
SEQ1M Q1T1	0.84	0.84
RND4K Q32T1	0.90	0.68
RND4K Q1T1	0.90	0.68

Рисунок 3.2.3 – Результаты производительности накопителя в проводном режиме

Заявленная скорость передачи данных у карт памяти класса скорости 10 составляет до 10 Мб/с, а скорость интерфейса USB версии 1.1, используемого у выбранного микроконтроллера, ограничивается 1.5 Мб/с [50; 51]. Результаты тестирования показали, что скорость передачи данных соответствует возможностям используемого носителя и аппаратным ограничениям микроконтроллера. При этом работа устройства оставалась стабильной, а ошибок файловой системы в процессе тестирования зафиксировано не было.

3.3 Тестирование беспроводного режима доступа

Тестирование беспроводного режима работы направлено на проверку встроенного веб-интерфейса, доступности устройства по сети, корректности работы веб-сервера и стабильности выполнения операций с файловой системой при беспроводном доступе.

Доступ к веб-интерфейсу осуществлялся с персональных компьютеров и мобильного устройства через веб-браузеры «Firefox» и «Google Chrome». Проверялась корректность отображения интерфейса, навигация по файловой системе, загрузка и скачивание файлов, а также выполнение управляющих операций. Пример отображения веб-страницы на компьютерах представлен на рисунке 3.3.1.

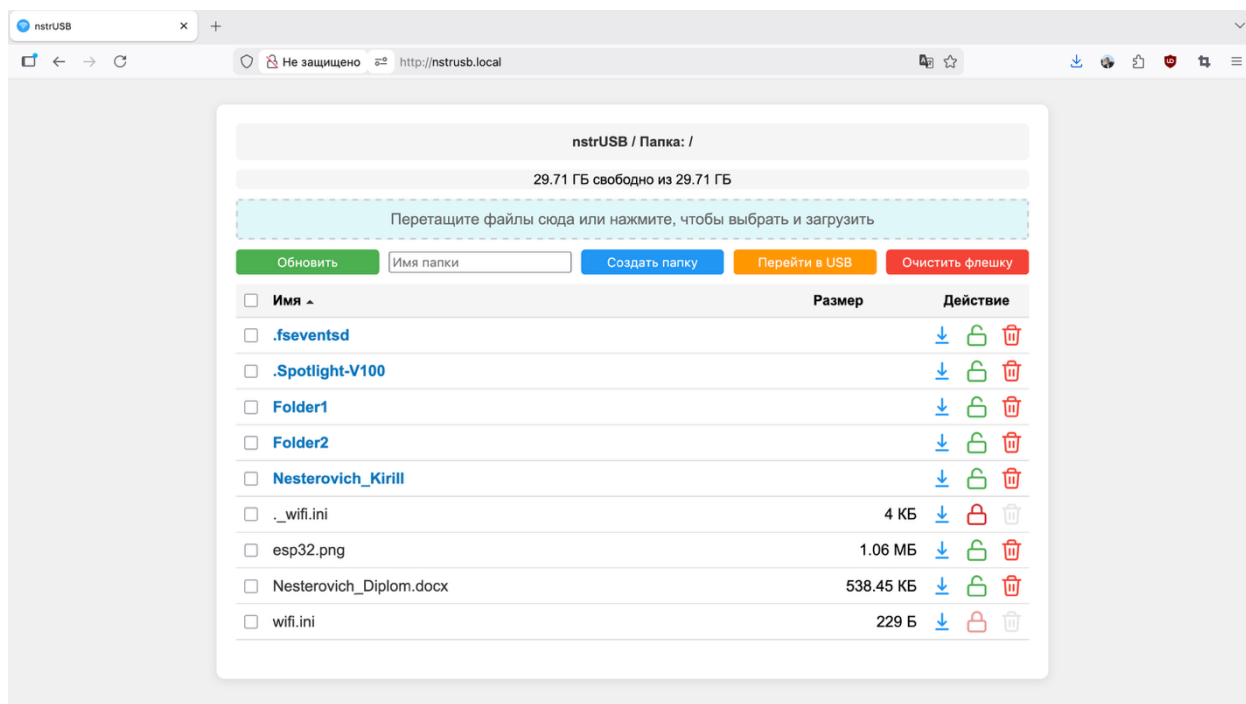


Рисунок 3.3.1 – Отображение веб-страницы с настольных устройств

Пример отображения веб-страницы на мобильных устройствах представлен на рисунке 3.3.2.

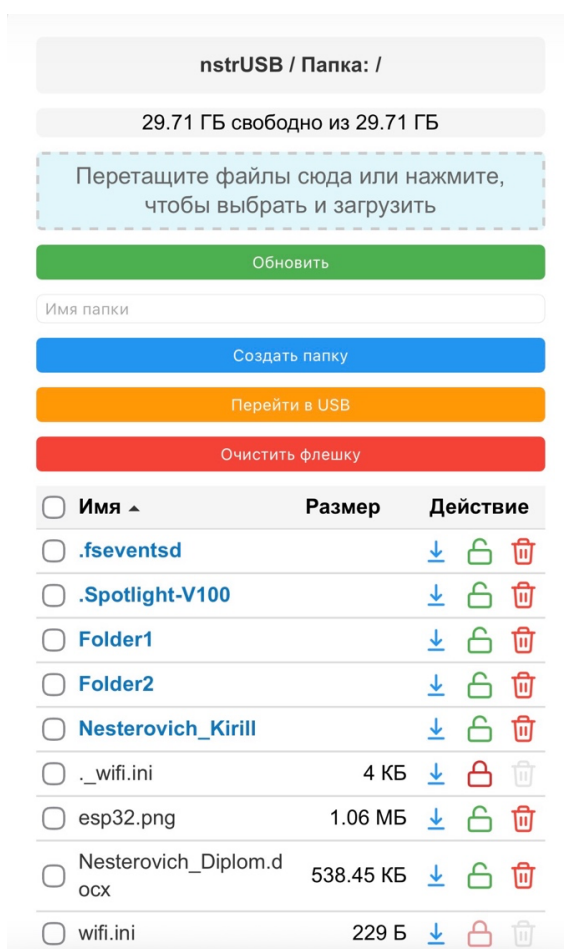


Рисунок 3.3.2 – Отображение веб-страницы на мобильных устройствах

Страница доступна как при указании назначенного маршрутизатором IP-адреса «172.20.10.5», так и при указании имени «nstrusb.local». Аналогичное поведение устройства наблюдалось при его работе в качестве точки доступа.

Следующим этапом необходимо проверить корректность загрузки файлов. Одновременно с этим имеет смысл протестировать возможность загрузки архива и последующей его распаковки, а также навигацию по каталогам. Загрузка файлов сопровождается соответствующей подсказкой с указанием текущей скорости загрузки. Вид страницы при загрузке файлов отображен на рисунке 3.3.3.

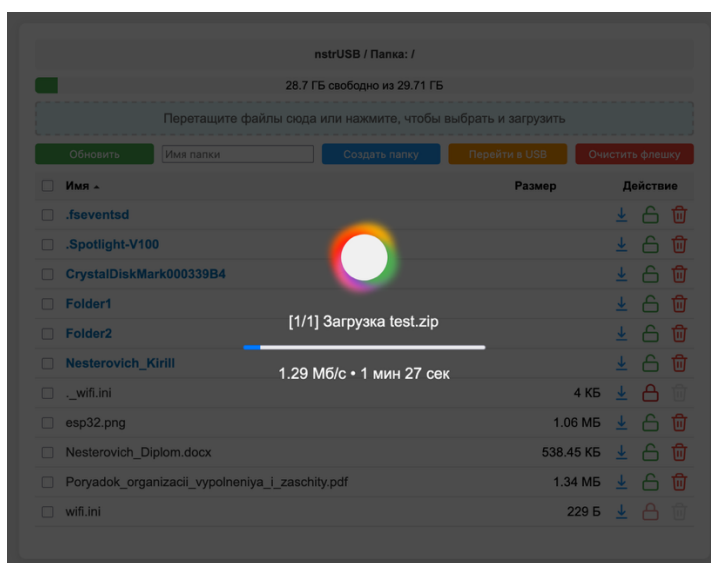


Рисунок 3.3.3 – Загрузка файлов в беспроводном режиме

Если загружаемый файл был архивом, показывается соответствующее уведомление о распаковке архива. Вид страницы при распаковке архива представлен на рисунке 3.3.4.

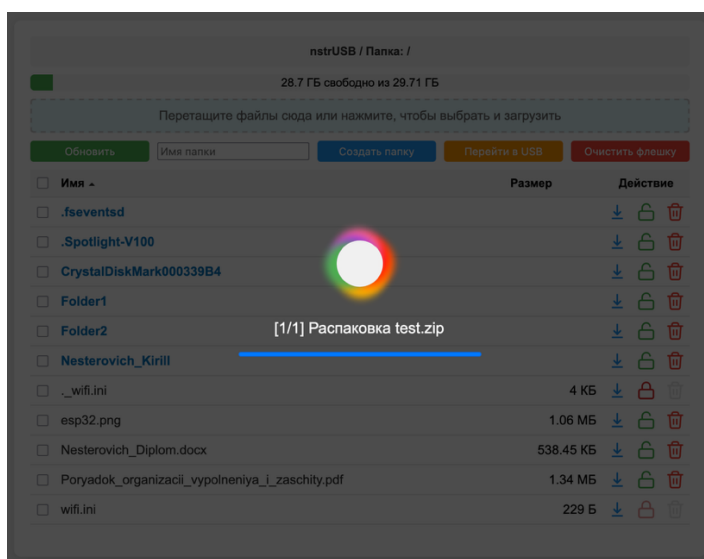


Рисунок 3.3.4 – Распаковка загруженного архива

При навигации по каталогам, путь и текущий каталог показывается в верхней части панели. Вернуться назад можно с помощью специальной кнопки с точками в верхней части списка файлов. Вид панели после навигации в определенную директорию представлен на рисунке 3.3.5.

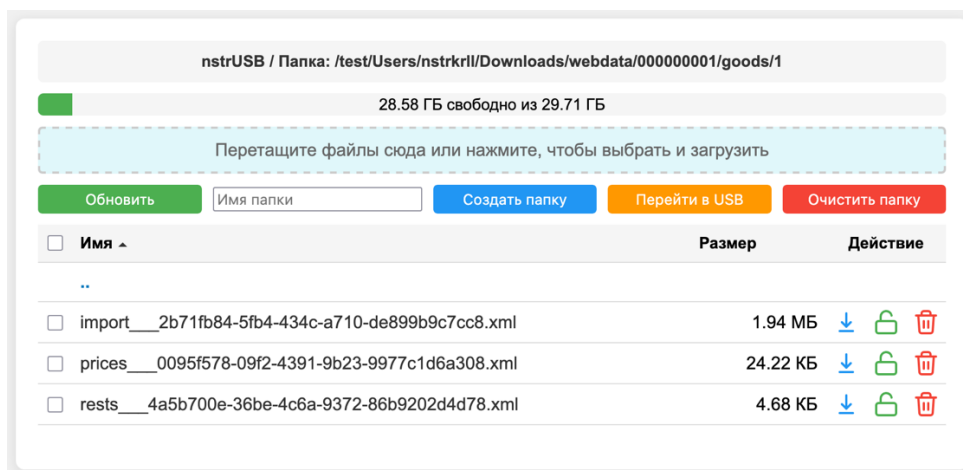


Рисунок 3.3.5 – Навигация по каталогам.

Также были протестированы функции удаления, блокирования и скачивания файлов и каталогов, создание нового каталога. Все тестируемые функции отвечают изначальным требованиям, ведут себя предсказуемо и сопровождаются соответствующими уведомлениями после выполнения. Вид уведомления представлен на рисунке 3.3.6.

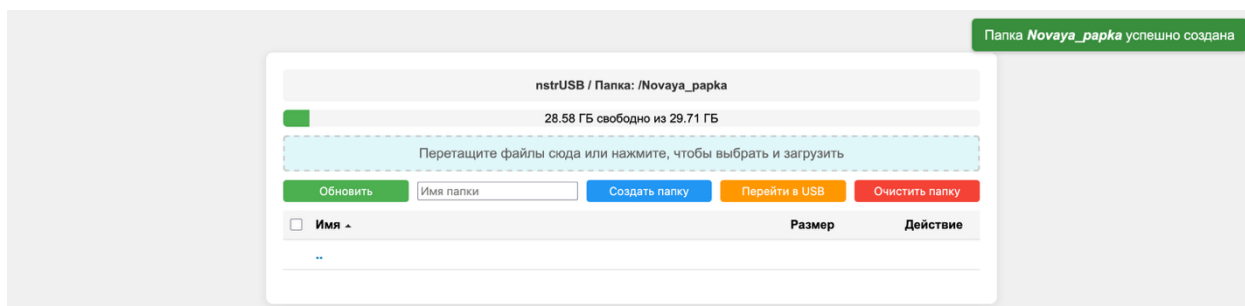


Рисунок 3.3.6 – Уведомление после выполнения операции в веб-панели

В ходе испытаний было подтверждено, что веб-интерфейс и его функции одинаково корректно функционируют в браузерах «Firefox» и «Google Chrome» на настольных системах, а также в мобильном браузере.

3.4 Тестирование переключения режимов работы

Один из наиболее критичных аспектов работы устройства – корректное переключение между проводным и беспроводным режимами доступа. В рамках этого этапа тестирования проверялась устойчивость системы при смене режимов работы и сохранность данных на файловой системе.

Переключение режимов выполнялось как вручную через веб-интерфейс, так и автоматически при отсутствии сетевой активности. В процессе тестирования контролировалось завершение текущих операций, корректное размонтирование файловой системы и отсутствие ошибок при повторном подключении устройства. При ручном переключении режимов, веб-страница отображала соответствующее уведомление, вид которого представлен на рисунке 3.4.1.

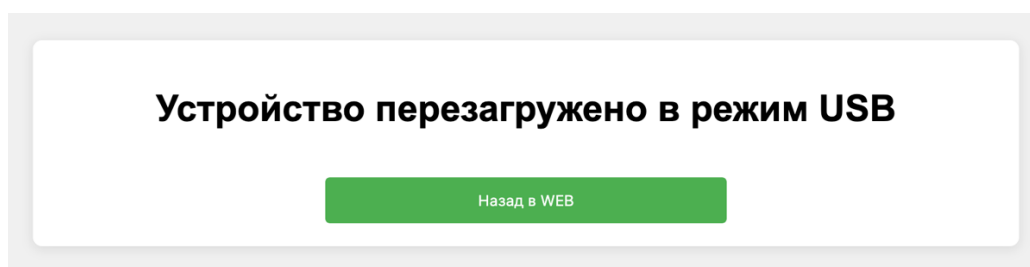


Рисунок 3.4.1 – Уведомление о переключении режима работы

Такое же сообщение увидят и другие пользователи, находящиеся на веб-странице в момент переключения режимов.

При активном использовании устройства в проводном режиме, например, при копировании файлов, и попытке перехода в беспроводной режим показывается соответствующее предупреждение. Вид уведомления представлен на рисунке 3.4.2.

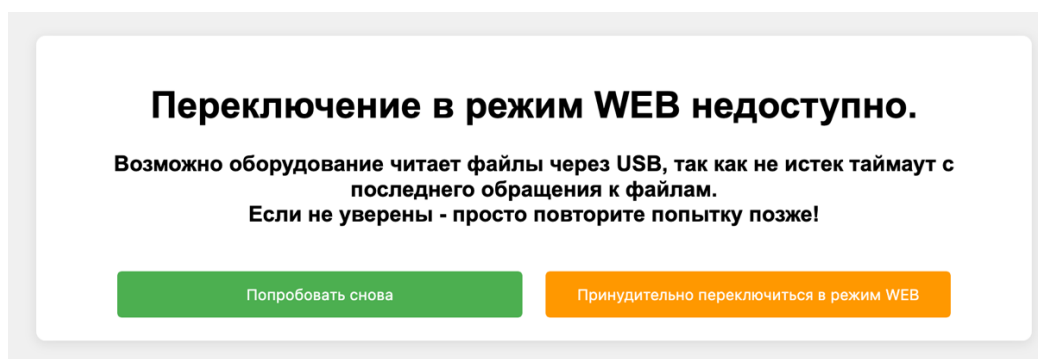


Рисунок 3.4.2 – Предупреждение об опасности переключения в беспроводной режим

Многократные попытки переключения режима, как и ожидается, не приводят к иному результату. Смена режима не доступна до тех пор, пока интерфейс USB не закончит свою работу. Такое поведение исключает повреждение файлов и повышает отказоустойчивость работы устройства.

Заключение

В рамках дипломной работы была рассмотрена задача проектирования и реализации автономного USB-накопителя с поддержкой облачного доступа к данным.

В ходе анализа предметной области было выявлено, что классические USB-накопители, несмотря на их широкое распространение и простоту использования, обладают рядом ограничений, связанных с жесткой привязкой к физическому подключению и отсутствием возможностей удаленного доступа. В то же время облачные и сетевые хранилища, предоставляют гибкость работы с данными, но зависят от внешней сетевой инфраструктуры и не всегда применимы в изолированных или ограниченных по доступу условиях. Анализ существующих решений и аналогов показал, что представленные на рынке устройства либо ориентированы на узкоспециализированные сценарии использования, либо обладают закрытой архитектурой и ограниченными возможностями модификации. Коммерческие накопители и Wi-Fi карты памяти не обеспечивают одновременное сочетание компактности, автономности и гибкости настройки. Использование одноплатных компьютеров позволяет реализовать сетевые хранилища, но такие решения избыточны по габаритам, энергопотреблению и стоимости.

На основании проведенного анализа была сформулирована цель дипломной работы, заключающаяся в разработке такого Wi-Fi USB-накопителя, который обеспечивал бы локальный доступ к данным через интерфейс USB и беспроводной доступ через веб-интерфейс, а так же не использовал внешние сервисы. Для достижения поставленной цели были определены задачи, охватывающие проектирование аппаратной и программной архитектуры, реализацию режимов работы устройства и проведение тестирования и оценки.

При выполнении работы была разработана аппаратная архитектура устройства на базе микроконтроллера ESP32-S3 в исполнении «Super Mini». Выбор данной платформы позволил реализовать поддержку интерфейсов USB и Wi-Fi без применения дополнительных средств или компонентов. В качестве носителя данных была использована карта памяти формата microSD. Это обеспечило достаточную производительность при чтении и записи файлов. Программная архитектура устройства была реализована в виде модульной встроенной прошивки с использованием фреймворка ESP-IDF. Это позволило разделить функциональные обязанности между отдельными подсистемами устройства. Централизованное управление режимами работы позволило правильно переключаться между проводным и беспроводным доступом без нарушения целостности данных.

Реализация режима «USB Mass Storage» позволила обеспечить интеграцию устройства в операционные системы хост-устройств. Накопитель определяется как стандартное внешнее устройство хранения данных и не требует установки дополнительных. Использование библиотеки «TinyUSB» и аппаратных возможностей микроконтроллера обеспечило стабильную работу проводного режима

Беспроводной режим работы устройства реализован на основе технологии Wi-Fi и встроенного веб-сервера. Использование веб-интерфейса позволило обеспечить удобный доступ к данным с любых клиентских устройств, оснащенных веб-браузером.

Важным элементом разработанной системы стал механизм управления конфигурацией устройства, основанный на использовании текстового файла настроек. Благодаря этому можно изменять параметры беспроводного подключения и режима его работы без перепрошивки.

Для обеспечения удобства сопровождения и поддержки устройства в процессе использования был реализован механизм обновления встроенного ПО. Обновление выполняется с использованием файла прошивки и не требует подключения к сети или применения специализированных средств, все происходит автоматически. Дополнительное использование шифрования файла прошивки позволяет повысить безопасность процесса обновления и защитить устройство от установки модифицированных версий программного обеспечения.

Тестирование разработанного прототипа проводилось в ручном режиме. Этап включал в себя проверку работы устройства в проводном и беспроводном режимах, а также оценку устойчивости системы при переключении между ними. Тестирование выполнялось на различных операционных системах и устройствах, что позволило оценить работоспособность накопителя в реальных сценариях эксплуатации. Результаты тестирования показали, что разработанный Wi-Fi USB-накопитель обеспечивает стабильную работу в заявленных режимах, правильно и стабильно обрабатывает операции чтения и записи данных и сохраняет целостность файловой системы при смене режимов доступа. Реализованные механизмы контроля активности интерфейсов позволили избежать конфликтного доступа к данным и повысить надежность работы устройства в целом.

Практическая значимость выполненной работы заключается в создании работоспособного прототипа автономного устройства хранения данных, которое может быть использовано в учебных, сервисных и инженерных задачах, а также в условиях ограниченного или отсутствующего сетевого подключения. Разработанное решение может служить основой для дальнейших доработок и расширения функциональности.

В перспективе дальнейшее развитие проекта может быть направлено на повышение производительности файловых операций и расширение возможностей веб-интерфейса. Можно рассмотреть возможность добавления аккумуляторной батареи в аппаратную схему для обеспечения полной автономности устройства и повышения степени его мобильности.

В ходе работы были успешно выполнены все поставленные задачи для создания прототипа Wi-Fi USB-накопителя:

- проанализирована предметная область автономных USB-накопителей с беспроводным доступом к данным;
- выполнен обзор существующих технических решений и аналогов;
- спроектирована аппаратно-программная архитектура Wi-Fi USB-накопителя;
- выбраны программные и аппаратные средства реализации устройства;
- разработано программное обеспечение для работы накопителя в режимах «USB Mass Storage» и Wi-Fi сервера;
- реализован веб-интерфейс для управления файловым хранилищем устройства;
- проведено тестирование разработанного прототипа и оценена его работоспособность.

Результаты исследования приняты к публикации в материалах II Международной научно-технической конференции «ERA – Современная наука: электроника, робототехника, автоматизация», Гомель, 30 – 31 октября 2025 г., а также опубликованы в сборниках материалов по итогам IX Всероссийской научно-технической конференции с международным участием «Современные информационные технологии в образовании и научных исследованиях», Донецк, 20 ноября 2025 г. [52] и XII Международной конференции аспирантов и молодых ученых «Молодежь XXI века: образование, наука, инновации», Витебск, 5 декабря 2025 г. [53].

Разработанное устройство внедрено в производственный процесс ОАО «Полоцкий молочный комбинат».

Список использованных источников

- 1 Axelson, J. USB Mass Storage: Designing and Programming Devices and Embedded Hosts : monogr. / J. Axelson. – Madison : Lakeview Research, 2006. – 1–34 p.
- 2 Buyya, R. Cloud Computing: Principles and Paradigms : monogr. / R. Buyya, J. Broberg, A. Goscinski. – Hoboken : John Wiley & Sons, 2011. – 43–56 p.
- 3 Mell, P. The NIST Definition of Cloud Computing : monogr. / P. Mell, T. Grance. – Gaithersburg : NIST Special Publication 800-145, 2011. – 2 p.
- 4 Ganssle, J. The Art of Designing Embedded Systems : monogr. / J.G. Ganssle . – Burlington : Elsevier, 2008. – 2–5 p.
- 5 ESP32 Wi-Fi & Bluetooth SoC : [site]. – URL: <https://www.espressif.com/en/products/socs/esp32> (date of access: 11.11.2025). – Text : electronic.
- 6 Mass Storage Class Specification Overview 1.4 : [site]. – URL: <https://www.usb.org/document-library/mass-storage-class-specification-overview-14> (date of access: 11.11.2025). – Text : electronic.
- 7 Axelson, J. USB Complete: The Developer's Guide : monogr. / J. Axelson. – Madison : Lakewiew Research, 2015. – 1–5 p.
- 8 Silberschatz, A. Operating System Concepts : monogr. / A. Silberschatz, P.B. Galvin, G. Gagne. – 9th ed. – Hoboken : John Wiley & Sons, 2013. – 489–505 p.
- 9 Coulouris, G. Distributed Systems: Concepts and Design : monogr. / G. Coulouris, J. Dollimore, T. Kindberg, G. Blair. – 5th ed. – Boston : Addison-Wesley, 2012. – 413–420 p.
- 10 Barr, M. Programming Embedded Systems : monogr. / M. Barr, A. Massa. – 2th ed. – O'Reilly, 2006. – 9–10 p.
- 11 Kurose, J. Computer Networking: A Top-Down Approach : monogr. / J. Kurose, K. Ross. – 7th ed. – Hoboken : Pearson, 2017. – 331–360 p.
- 12 Zhang, Q. Cloud computing: state-of-the-art and research challenges : monogr. / Q. Zhang, L. Cheng, R. Boutaba. – Waterloo : Internet Serv Appl, 2010. – 7 p.
- 13 What is FTP and why is it needed : [site]. – URL: <https://realhost.pro/en/blog/what-is-ftp> (date of access: 13.11.2025). – Text : electronic.
- 14 Fielding, R.T. Architectural Styles and the Design of Network-based Software Architectures : diss. / R.T. Fielding. – Irvine : University of California, 2000. – 67 p.
- 15 Tanenbaum, A.S. Distributed Systems: Principles and Paradigms : monogr. / A.S. Tanenbaum, M.V. Steen. – 2th ed. – New Jersey : Pearson, 2007. – 31 p.

16 Toshiba FlashAir W04 Wi-Fi SD card review : [site]. – URL: <https://ricksreviews.org/blog/2017/08/24/toshiba-flashair-w04-review/> (date of access: 17.11.2025). – Text : electronic.

17 Eye-Fi Mobi – карта памяти SD с Wi-Fi : [сайт]. – URL: <https://www.hardwareluxx.ru/index.php/artikel/consumer-electronics/gadgets/30539-eye-fi-mobi.html> (дата обращения: 17.11.2025). – Текст : электронный.

18 SanDisk Wireless Stick : [сайт]. – URL: <https://support.ru.wd.com/app/products/product-detailweb/session/L3RpbWUvMTc3MDMwNjIyNC9nZW4vMTc3MDMwNjIyNC9zaWQvZlVnaVdkeGFtNzRpTDFZc0h2NUhTSmpnUVBxNU9ISWxweXkwVVh0UDZINGplM0lNRU5qTVpqZVdWUzlaaHJ6ZTdXOE40U2ZuMVdqQml5Ukt4JTdFMWM4SnNER0VBMFNBZGZHc0Y0SzFicFlCYlhaWm1Ccm1SMk0xQnc1MjElMjE=/p/6790> (дата обращения: 18.11.2025). – Текст : электронный.

19 Обзор SanDisk Connect Wireless Stick: флэшка из будущего : [сайт]. – URL: <https://4pda.to/2015/10/24/253188/> (дата обращения: 18.11.2025). – Текст : электронный.

20 Wireless Flash Drive End of Support : [site]. – URL: https://support-en.sandisk.com/app/answers/detailweb/a_id/48977/~/~wireless-flash-drive-end-of-support (date of access: 18.11.2025). – Text : electronic.

21 Upton, E. Raspberry Pi User Guide : monogr. / E. Upton, G. Halfacree. – Chichester : John Wiley & Sons, 2016. – 13–14 p.

22 Koopman, P. Embedded System Design Issues : monogr. / P. Koopman. – Pittsburgh : Carnegie Mellon University, 2000. – 3 p.

23 Bass, L. Software Architecture in Practice : monogr. / L. Bass, P. Clements, R. Kazman. – 4th ed. – New Jersey : Addison-Wesley, 2021. – 45–51 p.

24 Douglass, B.P. Real-Time Design Patterns: Robust Scalable Architecture for Real-Time Systems : monogr. / B.P. Douglass. – New Jersey : Addison-Wesley, 2002. – 85–92 p.

25 FatFs – Generic FAT Filesystem Module : [site]. – URL: <https://elm-chan.org/fsw/ff/> (date of access: 08.12.2025). – Text : electronic.

26 Обзор протокола HTTP : [сайт]. – URL: <https://developer.mozilla.org/ru/docs/Web/HTTP/Guides/Overview> (дата обращения: 08.12.2025). – Текст : электронный.

27 ESP32-S3 Technical Reference Manual : [site]. – URL: https://documentation.espressif.com/esp32-s3_technical_reference_manual_en.pdf (date of access: 11.12.2025). – Text : electronic.

28 Wolf, M. Computers as Components: Principles of Embedded Computing System Design : monogr. / M. Wolf. – 3th ed. – Amsterdam : Morgan Kaufmann, 2012. – 17–19 p.

- 29 Espressif Systems. ESP32-S3 Datasheet : [site]. – URL: https://documentation.espressif.com/esp32-s3_datasheet_en.pdf (date of access: 11.12.2025). – Text : electronic.
- 30 ESP32-S3 Super Mini Development Board : [site]. – URL: <https://www.espboards.dev/esp32/esp32-s3-super-mini/> (date of access: 11.12.2025). – Text : electronic.
- 31 Secure Digital Multimedia Card (SDMMC) : [site]. – URL: <https://documentation.infineon.com/aurixtc3xx/docs/nit1715676525508> (date of access: 12.12.2025). – Text : electronic.
- 32 Интерфейс передачи данных SPI : [сайт]. – URL: <https://3d-diy.ru/blog/interfeys-peredachi-dannykh-spi/> (дата обращения: 12.12.2025). – Текст : электронный.
- 33 IEEE Standard for Information Technology : [site]. – URL: <https://thewifiiofthings.com/wp-content/uploads/2021/08/802.11-2020-Preview.pdf> (date of access: 12.12.2025). – Text : electronic.
- 34 Shaw, M. Software Architecture – Perspectives On An Emerging Discipline : monogr. / M. Shaw, D. Garlan. – New Jersey : Prentice-Hall, 1996 . – 27–32 p.
- 35 ESP-IDF Programming Guide : [site]. – URL: <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/index.html> (date of access: 17.12.2025). – Text : electronic.
- 36 Helihiy, M. The Art of Multiprocessor Programming : monogr. / M. Helihiy, N. Shavit. – Amsterdam : Morgan Kaufmann, 2008. – 3–15 p.
- 37 Serving Embedded Content via Web Applications: Model, Design and Experimentation : [site]. – URL: <https://scispace.com/pdf/serving-embedded-content-via-web-applications-model-design-4tt7pet7ui.pdf> (date of access: 17.12.2025). – Text : electronic.
- 38 Espressif's Additions to TinyUSB : [site]. – URL: https://components.espressif.com/components/espressif/esp_tinyusb/versions/2.1.0/readme (date of access: 22.12.2025). – Text : electronic.
- 39 TinyUSB Architecture : [site]. – URL: <https://docs.tinyusb.org/en/latest/reference/architecture.html> (date of access: 22.12.2025). – Text : electronic
- 40 Real-time operating system for microcontrollers and small microprocessors : [site]. – URL: <https://www.freertos.org/> (date of access: 22.12.2025). – Text : electronic.
- 41 Barry, R. Mastering the FreeRTOS Real Time Kernel : monogr. / R. Barry. – Cambridge : Real Time Engineers, 2016. – 2 p.
- 42 ESP32 Wi-Fi Driver : [site]. – URL: <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-guides/wifi.html> (date of access: 05.01.2026). – Text : electronic.

43 Многоадресный DNS (MDNS) в домашних сетях : [site]. – URL: <https://ip-calculator.ru/blog/networking/multicast-dns/> (дата обращения: 05.01.2026). – Текст : электронный.

44 ESP32 HTTP Server : [site]. – URL: https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/protocols/esp_http_server.html (date of access: 13.01.2026). – Text : electronic.

45 Using the miniz-cpp Library to Write and Read ZIP files : [site]. – URL: <https://www.informit.com/articles/article.aspx?p=3150354&seqNum=13> (date of access: 13.01.2026). – Text : electronic.

46 Introducing asynchronous JavaScript : [site]. – URL: https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Async_JS/Introducing (date of access: 14.01.2026). – Text : electronic.

47 Introducing JSON : [site]. – URL: <https://www.json.org/json-en.html> (date of access: 14.01.2026). – Text : electronic.

48 Over The Air Updates (OTA) : [site]. – URL: <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/system/ota.html> (date of access: 20.01.2026). – Text : electronic.

49 CrystalDiskMark – Drive speed benchmarking : [site]. – URL: <https://crystaldiskmark.org/> (date of access: 21.01.2026). – Text : electronic.

50 Руководство по классам скорости для карт памяти SD и microSD : [сайт]. – URL: <https://www.kingston.com/ru/blog/personal-storage/memory-card-speed-classes> (дата обращения: 21.01.2026). – Текст : электронный.

51 Теоретическая скорость интерфейса USB : [сайт]. – URL: https://support-ru.wd.com/app/answers/detailweb/a_id/10909/~/%D0%A2%D0%B5%D0%BE%D1%80%D0%B5%D1%82%D0%B8%D1%87%D0%B5%D1%81%D0%BA%D0%B0%D1%8F-%D1%81%D0%BA%D0%BE%D1%80%D0%BE%D1%81%D1%82%D1%8C-%D0%B8%D0%BD%D1%82%D0%B5%D1%80%D1%84%D0%B5%D0%B9%D1%81%D0%B0-usb (дата обращения: 21.01.2026). – Текст : электронный.

52 Проектирование USB-флешки с интегрированным Wi-Fi-сервером / К. А. Нестерович, П. В. Травничева. – Текст : электронный // Сайт конференции. – URL: <https://amai.fisp.donntu.ru/programma-sitoni-2025> (дата обращения: 22.01.2026). – Электрон. версия ст. из: Современные информационные технологии в образовании и научных исследованиях : материалы IX Всероссийской научно-технической конференции с международным участием, Донецк, 20 ноября 2025 г. Донецк : ФГБОУ ВО «ДонНТУ», 2025. – С. 150–151.

53 Разработка комбинированного USB-накопителя с беспроводным доступом к данным / К. А. Нестерович, П. В. Травничева. – Текст :

электронный // Репозиторий ВГУ им. П.М. Машерова. – URL: <https://rep.vsu.by/handle/123456789/49523> (дата обращения: 22.01.2026). – Электрон. версия ст. из: Молодежь XXI века: образование, наука, инновации : материалы XII Международной конференции аспирантов и молодых ученых, Витебск, 5 декабря 2025 г. Витебск : ВГУ имени П. М. Машерова, 2025. – Т. 1. – С. 29–30.

Список опубликованных научных работ
Нестеровича Кирилла Анатольевича

1. Проектирование USB-флешки с интегрированным Wi-Fi-сервером / К. А. Нестерович, П. В. Травничева // Современные информационные технологии в образовании и научных исследованиях : материалы IX Всероссийской научно-технической конференции с международным участием, Донецк, 20 ноября 2025 г. / редкол.: В. Н. Павлыш (гл. ред.) [и др.]. – Донецк : ФГБОУ ВО «ДонНТУ», 2025. – С. 150–151.
2. Разработка комбинированного USB-накопителя с беспроводным доступом к данным / К. А. Нестерович, П. В. Травничева // Молодежь XXI века: образование, наука, инновации : материалы XII Международной конференции аспирантов и молодых ученых, Витебск, 5 декабря 2025 г. / редкол.: Е. Я. Аршанский (гл. ред.) [и др.]. – Витебск : ВГУ имени П. М. Машерова, 2025. – Т. 1. – С. 29–30.

Содержание электронного носителя

На электронном носителе расположены следующие файлы:

- файл «Nesterovich_Diplom.pdf» содержит текст дипломной работы;
- Папка «Project» содержит исходный код проекта.